



École Nationale de l'Aviation Civile
7 Av. Édouard Belin
31400 Toulouse Cedex 9



**Institut de Recherche en Informatique de
Toulouse**
Cr Rose Dieng-Kuntz
31400 Toulouse



**Laboratoire d'Analyse et d'Architecture des
Systèmes**
7 Avenue du Colonel Roche
31400 Toulouse



**Artificial and Natural Intelligence Toulouse
Institute**
3 Rue Tarfaya
31400 Toulouse

Amélioration des méthodes de Benders pour un problème de planification robuste sous incertitude

Mathis FRANCINE-HABAS

Encadrants : Christian Artigues (Directeur de recherche au LAAS-CNRS)
Romain Guillaume (Enseignant-chercheur à l'IRIT / ANITI)

Tuteur : Mohammed Sbihi (Enseignant-chercheur à l'ENAC)

Soutenu le : 27 / 08 / 2026
2025-2026

Remerciements

Tout d'abord, je tiens à remercier toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et à la relecture de ce rapport.

Je remercie tout particulièrement Messieurs Christian Artigues et Romain Guillaume pour la confiance qu'ils m'ont accordée, ainsi que pour leur accompagnement tout au long de ce projet de recherche.

Je souhaite également remercier l'ensemble de mes professeurs pour la qualité de leur enseignement, qui m'a permis d'acquérir les connaissances nécessaires à la réussite de ma formation et de ce stage.

Enfin, ce stage ne se serait pas aussi bien déroulé sans la présence des différent·e·s chercheur·euse·s, doctorant·e·s et stagiaires avec qui j'ai partagé beaucoup de temps. Je remercie chaleureusement toute l'équipe ROC de m'avoir permis de travailler dans un cadre plus que bénéfique.

Table des matières

1	Introduction	1
2	Contexte de la recherche	2
3	Littérature	3
3.1	Définition et modélisation du problème	4
3.2	Méthode de Benders Adverse (BA)	4
3.2.1	Le problème MinMax	5
3.2.2	Le sous-problème Adv	6
3.2.3	L'algorithme de Benders Adverse	7
3.3	Méthode de Benders Adverse Étendue (BAE)	7
3.3.1	Le problème MinMax	8
3.3.2	Le sous-problème Adv	9
3.3.3	L'opération de fusion	10
3.3.4	Stratégies de sélection des pires scénarios	10
3.3.5	L'algorithme de Benders Adverse Étendue	11
4	Objectifs	12
4.1	Manipulation théorique et restructuration du modèle	12
4.2	Analyse des performances de référence	13
4.3	Définition de l'objectif principal	14
5	Propositions	15
5.1	L'heuristique <i>KCU</i>	16
5.1.1	Introduction	16
5.1.2	Extraction et factorisation	16
5.1.3	Algorithme glouton	16
5.1.4	Limites de l'approche <i>KCU</i>	17
5.2	L'heuristique <i>KCUD</i>	17
5.2.1	Introduction	17
5.2.2	Algorithme glouton	17
5.2.3	Limites de l'approche <i>KCUD</i>	18
5.3	L'heuristique <i>KCOG</i>	18
5.3.1	Introduction	18
5.3.2	Algorithme de recherche en profondeur (DFS)	19
5.3.3	Algorithme de calcul de distance	19
5.3.4	Algorithme de sélection MaxMin	20
5.3.5	Limites de l'approche <i>KCOG</i>	22
5.4	L'heuristique <i>KCOGL</i>	22
5.4.1	Introduction	22

5.4.2	Algorithme d'extraction aléatoire priorisé	22
5.4.3	Limites de l'approche <i>KCOGL</i>	22
5.5	L'heuristique <i>KCRDKL</i>	23
5.5.1	Introduction	23
5.5.2	Algorithme d'extraction aléatoire priorisée	23
5.5.3	Limites de l'approche <i>KCRDKL</i>	23
6	Résultats	24
6.1	Protocole expérimental	24
6.2	Expérience 1 : Apports de la compilation de connaissances ($N = 1$) . . .	24
6.3	Expérience 2 : Extremum temporel vs. extremum spatial ($N = 2$)	25
6.4	Expérience 3 : Apports de la distance ($N > 2$)	28
6.5	Expériences 4 et 5 : Limites de <i>BAE</i> et impact de la périodicité	30
6.6	Validation de nos modèles par la méthode de Monte-Carlo	32
6.6.1	Protocole expérimental	32
6.6.2	Analyse de la robustesse	33
6.7	Conclusion & Pistes d'améliorations	33
7	Conclusion	36
7.1	Synthèse sur le travail effectué	36
7.2	Apports professionnels	36
7.3	Apports techniques	36
7.4	Perspectives	37
	Annexes	40
A	Définition d'un graphe de budget	41
B	Définition d'un graphe partiel de budget	42
C	Algorithme de programmation dynamique du problème Adv (<i>BAE</i>)	43
D	Algorithme d'initialisation du budget d'incertitude	44
E	Algorithme d'extraction du scénario optimal (<i>KCU</i>)	45
F	Algorithmes d'extraction des scénarios optimaux (<i>KCUD</i>)	46
G	Algorithme récursif d'extraction des scénarios (<i>DFS</i>)	48
H	Schéma du fonctionnement de l'algorithme <i>DFS</i>	49
I	Algorithme de discrétisation MaxMin (<i>KCOG</i>, <i>KCOGL</i>)	50
J	Algorithme complet d'extraction (<i>KCOGL</i>)	51

K	Algorithme d'extraction priorisé (KCRDKL)	52
L	Analyse de la robustesse d'un plan de production par Monte Carlo	53
M	Déroulement de l'algorithme de Benders Adverse	54
	M.1 Déroulement de l'algorithme	54
	M.1.1 Formulation du problème	54
	M.1.2 Itérations	54
	M.2 Résultats de l'expérimentation par CPLEX	57
N	Reproducing Robust Instance Scheduling	58
	N.1 Launch of the Experiment	58
	N.2 Test Different Heuristics	59
	N.3 Instance Generation	60
	N.4 Statistical Analysis of the Results	61

Table des figures

1	Schéma de la méthode de Benders Adverse. [14]	5
2	Modélisation du problème de <i>Lot-Sizing</i> . [13]	6
3	Schéma générique du graphe de budget. [8]	7
4	Reformulation du problème Maître. [13]	8
5	Sélection des deux scénarios optimaux les plus distants temporellement lors de la phase Adv de <i>backtracking</i>	18
6	Définition de la métrique de distance.	21
7	Schéma de fonctionnement de la discrétisation MaxMin	22
8	Comparaison des temps de résolution du sous-problème et du problème maître pour <i>BA</i> & <i>KCU</i>	25
9	Analyse comparative entre <i>BA</i> & <i>KCU</i>	26
10	Comparaison des temps de résolution du sous-problème et du problème maître pour <i>KCUD</i> , <i>KCOGL</i> & <i>KCRDKL</i>	26
11	Analyse comparative entre <i>KCUD</i> & <i>KCOGL</i>	27
12	Accélération de la distribution entre <i>KCRDKL</i> & <i>KCOGL</i>	28
13	Progression lissée entre <i>KCRDKL</i> & <i>KCOGL</i> par rapport à <i>N</i>	29
14	Impact de <i>N</i> et <i>L</i> sur les performances.	29
15	Comparaison des temps de résolution du sous-problème et du problème maître avec périodicité.	30
16	Comparaison des temps de résolution des sous-problèmes et du pro- blème Maître sans périodicité.	31
17	Schéma du fonctionnement de l'algorithme DFS.	49
18	Graphe de budget symbolisant l'expérience.	54
19	Résolution de l'instance par CPLEX via la méthode de Benders Adverse.	57

Liste des tableaux

1	Extrait du temps moyen et du nombre moyen d'itérations entre BA , BAE_{All} & BAE_{Few} pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$	14
2	Extrait du temps moyen et du nombre moyen d'itérations des 6 méthodes pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$	31
3	Analyse de la robustesse d'un plan de production par la méthode de Monte-Carlo (KCU , $KCUD$, $KCRDKL$ et $KCOGL$).	33
4	Analyse de la robustesse d'un plan de production par la méthode de Monte Carlo (BA , BAE_{All} et BAE_{Few}).	53
5	Simulation parameters settings.	58
6	Instance generation settings.	60

List of Algorithms

1	Adversarial Benders Method	7
2	Extended Adversarial Benders Method	11
3	Calculate Manhattan Distance (<i>KCOG</i> , <i>KCOGL</i>)	21
4	Adv_subGraph	43
5	Proportional Initialization of Uncertainty Bounds	44
6	Greedy Extraction of the Optimal Scenario (<i>KCU</i>)	45
7	Extraction of the Optimal Paths (<i>KCUD</i>)	46
8	Configurable Reporting Function	47
9	Recursive Extraction of Scenarios (DFS) (<i>KCOG</i> , <i>KCOGL</i> , <i>KCRDK</i> , <i>KCRDKL</i>)	48
10	MaxMin Selection (<i>KCOG</i> , <i>KCOGL</i>)	50
11	Extraction of N Scenarios via L_1 -Distance (<i>KCOGL</i>)	51
12	Extraction of N Scenarios via Prioritized Random Sampling (<i>KCRDKL</i>) .	52

1 Introduction

Historiquement dérivée des stratégies militaires de la Seconde Guerre mondiale, la recherche opérationnelle (RO) s'est peu à peu affirmée comme une discipline scientifique essentielle pour l'allocation optimale des ressources et l'amélioration des processus industriels. Son efficacité actuelle repose sur trois axes. Le premier consiste à modéliser mathématiquement ces processus complexes afin de traduire fidèlement les contraintes et les objectifs. Le deuxième se concentre sur le développement de méthodes de résolution capables d'être appliquées à grande échelle. Enfin, le dernier levier de performance, devenu central aujourd'hui, est le traitement intégré et intelligent des données.

C'est précisément sur ce dernier point que la recherche opérationnelle converge vers l'intelligence artificielle. Cependant, l'exploitation de ces données provenant d'environnements réels confronte systématiquement les modèles à un défi majeur : l'omniprésence de l'incertitude. En effet, l'instabilité de la demande, des prix et du climat complique la planification au quotidien. La production, la logistique et les approvisionnements sont soumis à des risques accrus, amplifiés par l'interdépendance des chaînes mondiales. Les modèles classiques supposent souvent un monde « parfait ». Optimiser ce scénario crée une stratégie fragile qui ne tiendra pas face au premier aléa. L'enjeu n'est donc plus seulement d'optimiser un plan idéal, mais aussi d'anticiper les écarts malgré ces incertitudes : c'est la naissance de l'optimisation robuste (OR).

A. L. Soyster a formalisé ce nouveau concept en 1973 dans [15]. Son innovation protège une solution contre le pire des cas. Elle suppose que chaque paramètre peut prendre simultanément sa valeur la plus défavorable. Cette nouvelle approche constitue le début de la prise de décision dans l'incertitude. Aujourd'hui, nous décrivons l'incertitude à l'aide de mesures et de scénarios critiques identifiés. Les applications de l'optimisation robuste englobent notamment la *supply chain*, la maintenance et la production, où les aléas locaux peuvent provoquer des effets systématiques sur les performances globales. Pour résoudre ces problèmes liés à l'optimisation robuste, plusieurs approches existent. L'une d'elles, connue sous le nom de « plans coupants » (ou *cutting-set*), a été introduite par [11]. Elle a ensuite été formalisée sous le nom de méthode de Benders Adverse (BA) par [13]. C'est cette approche que nous avons utilisée et que nous détaillerons par la suite.

Comment les méthodes de Benders peuvent-elles optimiser la planification robuste face à l'incertitude ? Nous commencerons par présenter le contexte de notre recherche, ferons un point sur les avancées scientifiques dans ce domaine, avant d'exposer nos objectifs, nos solutions ainsi que les résultats obtenus. Les contributions techniques et professionnelles issues de cette expérience seront abordées dans une dernière partie.

2 Contexte de la recherche

Ce stage s'inscrit au cœur d'un écosystème scientifique toulousain riche, porté par une collaboration entre trois acteurs majeurs de la recherche en intelligence artificielle et en informatique : l'institut ANITI, l'IRIT-CNRS et le LAAS-CNRS.

L'impulsion de cette dynamique est notamment portée par la chaire HEROIC¹ d'ANITI (*Artificial and Natural Intelligence Toulouse Institute*), un cluster labellisé par la communauté d'universités et d'établissements de Toulouse. Son ambition est de concevoir des systèmes d'intelligence artificielle dont les prises de décision sont explicables, équitables, fiables et robustes, une condition indispensable pour permettre leur déploiement dans des secteurs critiques à haut risque.

Cette volonté de développer des systèmes décisionnels de confiance s'appuie sur l'expertise de deux laboratoires. D'une part, l'Institut de Recherche en Informatique de Toulouse (IRIT), qui est un laboratoire de recherche publique rattaché au Centre National de la Recherche Scientifique (CNRS). D'autre part, le Laboratoire d'Analyse et d'Architecture des Systèmes (LAAS) est une unité propre du CNRS. Si le LAAS aborde des thématiques très variées, allant de la robotique aux micro-nano-bio-technologies, il se distingue particulièrement par sa recherche théorique et méthodologique sur la conception de lois mathématiques appliquées à la commande et à la décision.

C'est à l'intersection de ces institutions, et plus spécifiquement au sein de l'équipe ROC (Recherche sur des problèmes d'Optimisation Combinatoire) du LAAS que s'est déroulé mon stage. Positionnée à la frontière entre la recherche opérationnelle et l'intelligence artificielle, cette équipe offre un cadre scientifique idéal pour aborder les problématiques d'optimisation robuste qui font l'objet de ce rapport.

Plus précisément, les travaux menés s'inscrivent dans la continuité directe de la thèse de Tom Portoleau (voir [13]), soutenue au sein de cette même équipe. L'objectif est d'approfondir et de consolider la méthode de Benders Adverse Étendue (BAE) qu'il a introduite, en surmontant ses limites combinatoires actuelles. À terme, les modèles d'optimisation et les heuristiques développés ont pour ambition d'être transférés vers l'industrie aéronautique, en partenariat avec Airbus. L'exploitation de ces algorithmes sur des données réelles permettra de tester leur robustesse et leur passage à l'échelle face à des problématiques concrètes et critiques de planification.

1. *Hybridizing Learning, seaRch and combinatorial Optimization for Industrial deCision-making.*

3 Littérature

L'optimisation robuste absolue (ou approche **MinMax**) de problèmes de planification est complexe. Pour contourner cette difficulté, les auteurs de [2] ont mis au point la méthode dite de Benders Adverse, qui s'appuie sur une décomposition itérative : elle isole un problème Maître, qui planifie la production, et un problème Adverse, qui évalue la robustesse de cette planification en cherchant le pire scénario possible.

Dans la littérature, [12] présente une variante de cette méthode où les pires scénarios² sont calculés de manière approximative tout en garantissant la convergence de l'algorithme. Plus récemment, les travaux de [14], sur lesquels s'appuie l'essentiel de notre étude, proposent la méthode de Benders Adverse Étendue, que nous noterons *BAE*³. Contrairement à l'approche classique, l'adversaire ne renvoie plus un unique pire cas, mais un ensemble de scénarios défavorables afin d'accélérer la convergence globale.

Pour justifier l'efficacité de cette approche étendue, il est nécessaire de s'appuyer sur le paradigme de la compilation de connaissances (*Knowledge Compilation* - *KC*). Initialement issue de l'intelligence artificielle et de l'informatique théorique [6], la *KC* consiste à compiler une théorie propositionnelle complexe (ici, l'espace des scénarios incertains) dans un langage cible qui est structuré, typiquement un Graphe Orienté Acyclique (DAG).

Comme démontré par Darwiche et Marquis [6], l'intérêt majeur des DAGs réside dans leur capacité à trouver un compromis idéal entre la précision et la tractabilité. En imposant des propriétés structurelles telles que la décomposabilité (où les sous-problèmes traitent de variables disjointes), le DAG permet de factoriser les redondances présentes dans une formulation de type CNF (*Conjunctive Normal Form*). Cette factorisation rend possible l'utilisation d'algorithmes permettant l'agrégation de scénarios en temps polynomial.

Dans le domaine de l'optimisation combinatoire (OC), cette logique a été transposée aux diagrammes de décision [3]. Dans la méthode classique de Benders, l'énumération itérative de scénarios discrets se traduit par l'ajout de contraintes linéaires indépendantes, potentiellement redondantes, quiaturent rapidement le problème Maître. À l'inverse, l'approche *BAE* proposée par [13] exploite la décomposabilité du DAG pour agréger ces scénarios défavorables. Au lieu de stocker chaque scénario individuellement, la méthode factorise leurs sous-chemins communs au sein d'une structure unique (que nous définirons par la suite comme le graphe partiel de budget⁴). Cette factorisation réduit le nombre de variables et de contraintes générées à chaque itération. Cela permet ainsi de limiter le phénomène d'explosion combinatoire tout en conservant l'information nécessaire et suffisante.

2. Dans la suite du rapport, les notions de « pire scénario », « pire chemin », « chemin critique optimal » et « scénario/chemin optimal » sont équivalentes.

3. L'auteur introduisant la méthode *BAE*, dans [13], a utilisé la notation *KC* pour *Knowledge Compilation*, mais dans notre étude, *KC* signifiera le paradigme de la compilation de connaissances.

4. Voir Définition A.1.

Dans la continuité de cette recherche, nous présenterons cette méthode pour un problème de dimensionnement de lots avec des contraintes de capacité (*Capacitated Lot-Sizing Problem - CLSP*), soumis à un budget d'incertitude portant sur la demande cumulée.

3.1 Définition et modélisation du problème

Afin de modéliser ce problème, nous considérons un horizon de planification divisé en T périodes. À chaque période $t \leq T$, la demande est notée d_t et la quantité produite x_t . Un plan de production est constitué par l'ensemble de ces variables décisionnelles sur l'horizon. Soit c^P le coût unitaire de lancement de production et c^I le coût unitaire de stockage appliqué au surplus de production (noté I_t). c^B représente le coût unitaire de pénalité de retard appliqué à la demande insatisfaite (notée B_t), tandis que b^P désigne le prix de vente unitaire appliqué à la quantité de demande effectivement satisfaite (notée s_t). Contrairement aux approches classiques où l'incertitude porte sur la demande marginale de façon indépendante, notre modèle intègre une incertitude sur la demande cumulée. Soient $X_t = \sum_{i=1}^t x_i$ la production cumulée et $D_t = \sum_{i=1}^t d_i$ la demande cumulée à l'instant t . L'incertitude sur la demande cumulée est modélisée par un intervalle symétrique $[\hat{D}_t \pm \Delta_t]$, où $\hat{D}_t \geq \Delta_t \geq 0$ représente la valeur nominale et Δ_t la déviation maximale. L'ensemble des variables de production cumulées, noté $X_t = (X_1, \dots, X_T)$, prend ses valeurs dans un polytope $\mathcal{X}$ de dimension T . Cette formulation sous forme de polytope permet de représenter non seulement des contraintes de capacité maximale, mais également d'autres relations linéaires entre les variables de production des différentes périodes.

Nous définissons un **scénario** comme une réalisation spécifique de ces incertitudes au sein d'un ensemble de scénarios possibles $\mathcal{U}$. En imposant une limite globale sur les déviations via un budget d'incertitude Γ , l'ensemble continu des scénarios admissibles se définit par l'Équation (3.1). Notons $\mathcal{D} = (D_t)_{t \leq T}$ le vecteur de la demande cumulée incertaine et $\hat{\mathcal{D}} = (\hat{D}_t)_{t \leq T}$ le vecteur de la demande cumulée nominale.

$$\mathcal{U} = \{ \forall t \in [1, T-1], (D_t)_{t \leq T} \mid D_t \leq D_{t+1}, D_t \in [\hat{D}_t - \Delta_t, \hat{D}_t + \Delta_t], \|\mathcal{D} - \hat{\mathcal{D}}\|_1 \leq \Gamma \} \quad (3.1)$$

Ce problème de *Lot-Sizing* déterministe sous-jacent étant *NP-Difficile* [5], sa résolution sous incertitude requiert une approche **MinMax**.

3.2 Méthode de Benders Adverse (BA)

Le principe fondamental de la méthode de Benders Adverse est de résoudre le problème **MinMax** de façon itérative. Plutôt que de résoudre le problème face à l'infinité de scénarios de $\mathcal{U}$ simultanément, le problème Maître est initialement résolu face à un scénario unique, correspondant à la demande nominale notée $\hat{\mathcal{D}}$.

Une fois le plan de production initial S obtenu, une phase adverse (le sous-problème **Adv**) est exécutée. Son rôle est d'identifier, pour ce plan S figé, le scénario $\mathcal{D}'$ maximisant les coûts. Ce pire scénario est alors ajouté à l'ensemble restreint des scénarios considérés, et le problème Maître est résolu à nouveau. Ce processus itératif, illustré par la Figure 1, s'arrête si le critère d'arrêt est atteint, c'est-à-dire lorsque l'adversaire n'est plus en mesure de trouver un scénario dégradant la valeur objective du problème Maître. Le Maître et l'adversaire ont alors convergé vers la même solution robuste.

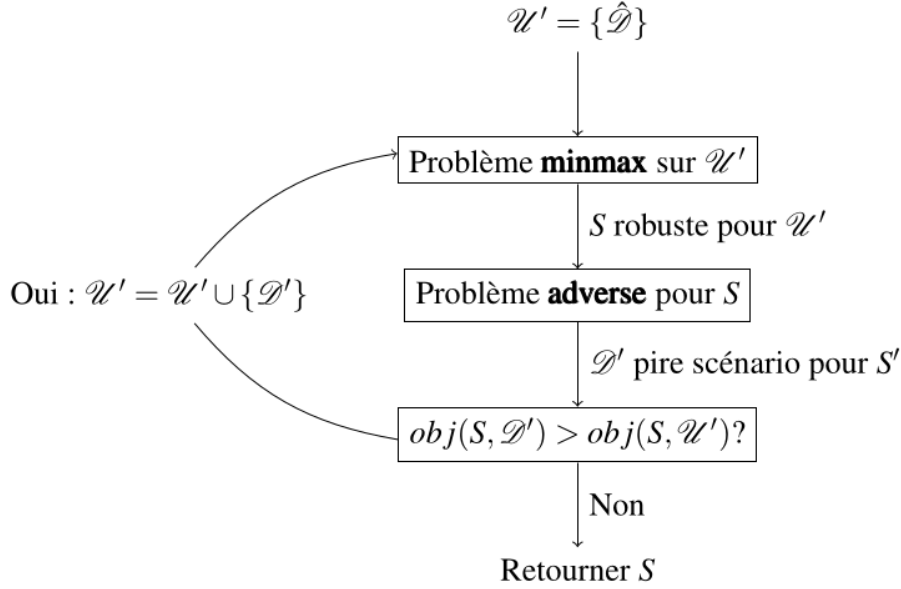


FIGURE 1 – Schéma de la méthode de Benders Adverse. [14]

Pour appliquer cette méthode à notre problème, il faut donc être capable de résoudre le problème **MinMax** pour une liste de scénarios donnée et de résoudre le problème **Adv** pour une solution du problème donné.

3.2.1 Le problème MinMax

L'objectif du problème Maître est de trouver le plan de production minimisant les coûts face au pire des scénarios connus. Mathématiquement, il s'agit de résoudre $\min_{X \in \mathcal{X}} \max_{u \in U} \text{Coût}(X, u)$, où $U \subseteq \mathcal{U}$ est la liste finie des scénarios générés par l'adversaire.

Afin de le modéliser sous la forme d'un programme linéaire en nombres entiers mixtes (MILP), une technique de reformulation en épigraphe est employée. On introduit une variable auxiliaire continue z dont le rôle est de majorer le coût total pour l'ensemble des scénarios $\xi \in U$. En minimisant ce majorant z , on minimise le coût du scénario le plus défavorable. Outre z , le modèle utilise une variable binaire $y_t \in \{0, 1\}$ indiquant si une production est lancée à la période t , ainsi que la constante M du *Big-M* pour lier les variables continues et binaires.

L'Inégalité (3.3) impose que la variable continue z soit supérieure ou égale au coût

$$\min \quad z \quad (3.2)$$

$$\text{s.t.} \quad \sum_{t=1}^T \left(c^I I_{t,\xi} + c^B B_{t,\xi} + c^P y_t - b^P s_{t,\xi} \right) \leq z \quad \forall \xi \in \mathcal{U}, \quad (3.3)$$

$$B_{t,\xi} - I_{t,\xi} = D_t^\xi - X_t \quad \forall t \in [T], \forall \xi \in \mathcal{U}, \quad (3.4)$$

$$\sum_{i=1}^t s_{i,\xi} = D_t^\xi - B_{t,\xi} \quad \forall t \in [T], \forall \xi \in \mathcal{U}, \quad (3.5)$$

$$X_t - X_{t-1} \leq M y_t \quad 2 \leq t \leq T, \quad (3.6)$$

$$X_1 \leq M y_1, \quad (3.7)$$

$$B_{t,\xi}, I_{t,\xi}, s_{t,\xi} \geq 0 \quad \forall t \in [T], \forall \xi \in \mathcal{U}, \quad (3.8)$$

$$y_t \in \{0, 1\} \quad \forall t \in [T], \quad (3.9)$$

$$X \in \mathcal{X} \subseteq \mathbb{R}_+^T \quad (3.10)$$

 FIGURE 2 – Modélisation du problème de *Lot-Sizing*. [13]

total d'exploitation, et ce, pour l'intégralité des scénarios adverses $\xi \in \mathcal{U}$. En minimisant z dans la fonction objectif, *CPLEX* minimise le coût associé au pire scénario de l'ensemble $\mathcal{U}$. La Contrainte (3.4) garantit la cohérence du système. À chaque période t et pour chaque scénario ξ , la différence entre les commandes en retard ($B_{t,\xi}$) et le stock physique ($I_{t,\xi}$) doit correspondre à l'écart entre la demande cumulée (D_t^ξ) et la production cumulée (X_t). Si la demande excède la production, le modèle génère une rupture; sinon, il génère du stock. La Contrainte (3.5) relie les ventes effectives à la demande. La somme des quantités vendues jusqu'à l'instant t doit être exactement égale à la demande cumulée moins les quantités non satisfaites ($B_{t,\xi}$). Les Contraintes (3.6) et (3.7) assurent la liaison entre la variable de production continue et la variable de décision binaire y_t . La production marginale à l'instant t ($X_t - X_{t-1}$) est majorée par $M y_t$. Cela signifie que, si une production est lancée, alors le solveur fixe $y_t = 1$ et force le coût de lancement c^P dans la fonction objectif. Enfin, les dernières contraintes donnent les domaines de définition des variables de décision.

3.2.2 Le sous-problème Adv

Une fois le plan de production X fixé, le sous-problème cherche à construire le pire scénario entier. Bien que ce problème de maximisation ait été démontré *NP*-difficile au sens faible par les auteurs de [8], ceux-ci ont également montré qu'il pouvait être résolu en temps pseudo-polynomial. L'astuce, qu'ils ont introduit dans la Définition A.1 (située en Annexe A), consiste à modéliser l'espace des déviations de la demande sous la forme d'un graphe acyclique orienté, au sein duquel la recherche du pire scénario se réduit à celle du plus long chemin.

De ce fait, évaluer un scénario revient à sommer les poids des arcs traversés, et identifier le pire scénario revient à extraire le chemin de poids maximal dans $\mathcal{G}$ (sché-

maté par la Figure 3). Pour la suite du rapport, le terme de « chemin » signifiera une suite d'arcs dans le graphe entre la source v_0 et le puits v_{T+1} .

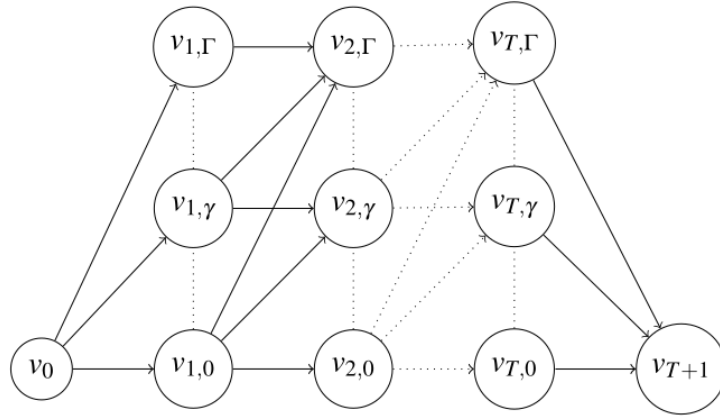


FIGURE 3 – Schéma générique du graphe de budget. [8]

3.2.3 L'algorithme de Benders Adverse

Voici l'algorithme implémenté par [13] pour résoudre le problème de Benders Adverse présenté dans la Figure 1. Tant que l'adversaire parvient à construire un graphe de budget dont le plus long chemin révèle un coût v' strictement supérieur au coût v anticipé par le Maître, ce nouveau scénario est ajouté à $\mathcal{U}$ et le processus se réitère.

Algorithm 1: Adversarial Benders Method

```

1  $\mathcal{U} \leftarrow \{\hat{\mathcal{D}}\}$ 
2  $(X, v) \leftarrow \text{SolveMaster}(U)$ 
3  $(\mathcal{D}, v') \leftarrow \text{SolveAdversarial}(X)$ 
4 while  $v \neq v'$ 
5    $v \leftarrow v'$ 
6    $\mathcal{U} \leftarrow \mathcal{U} \cup \{\mathcal{D}\}$ 
7    $(X, v) \leftarrow \text{SolveMaster}(U)$ 
8    $(\mathcal{D}, v') \leftarrow \text{SolveAdversarial}(X)$ 
9 return  $(X, v)$ 
```

3.3 Méthode de Benders Adverse Étendue (BAE)

Cette section détaille la méthode de Benders Adverse Étendue, formellement introduite par [13]. Cette approche constitue le fondement de mes recherches. Il s'agit d'une extension de la méthode classique de Benders Adverse présentée précédemment. Elle a été conçue pour accélérer la convergence de l'algorithme. Plutôt que d'isoler le pire scénario à chaque itération du sous-problème **Adv**, la BAE génère un ensemble de

scénarios défavorables. L'enjeu de cette méthode réside dans la résolution du problème Maître : sa formulation initiale, basée sur une liste explicite de scénarios $\mathcal{U}$, souffre d'une explosion combinatoire et s'adapte très difficilement à l'échelle lorsque ce nombre augmente. Pour faire face à cette limite, le problème Maître est redéfini en s'appuyant sur une structure plus compacte : le graphe partiel de budget.

3.3.1 Le problème MinMax

Pour ce faire, les auteurs de [13] ont précisément défini cette notion (voir la Définition B.1 située en Annexe B). Lorsque le problème Maître du Benders Adverse classique se basait sur l'approximation de l'ensemble des scénarios $\mathcal{U}$ par une liste de scénarios. Cette fois-ci, il se fait par l'extraction d'un sous-graphe issu du graphe de budget global (voir la Figure 3).

Grâce à cette structure, le problème Maître peut être reformulé comme un problème de calcul du plus long chemin dans le graphe partiel $\mathcal{G}'$. Cette formulation s'appuie sur des variables duales, notées $\pi_{t,i}$, qui représentent le coût du plus long chemin depuis le nœud source v_0 jusqu'au nœud $v_{t,i}$. Le programme linéaire en variables mixtes modélisant ce nouveau problème Maître s'écrit alors de la manière suivante :

$$\min \quad \pi_{T+1} \quad (3.11)$$

$$\text{s.t.} \quad \pi_{t,j} - \pi_{(t-1),i} \geq c^I (X_t - (\hat{\mathcal{D}}_t - (j - i))) + c^P y_t, \quad t \in [1, T - 1], a_{t,i,j} \in A', \quad (3.12)$$

$$\pi_{t,j} - \pi_{(t-1),i} \geq c^B (\hat{\mathcal{D}}_t + (j - i) - X_t) + c^P y_t, \quad t \in [1, T - 1], a_{t,i,j} \in A', \quad (3.13)$$

$$\pi_{t,j} - \pi_{(t-1),i} \geq c^I (X_t - (\hat{\mathcal{D}}_t - (j - i))) + c^P y_t - b^P (\hat{\mathcal{D}}_t - (j - i)), \quad t = T, a_{t,i,j} \in A', \quad (3.14)$$

$$\pi_{t,j} - \pi_{(t-1),i} \geq c^B (\hat{\mathcal{D}}_t + (j - i) - X_t) + c^P y_t - b^P X_t, \quad t = T, a_{t,i,j} \in A', \quad (3.15)$$

$$\pi_{T+1} - \pi_{T,i} \geq 0, \quad 0 \leq i \leq \Gamma, \quad (3.16)$$

$$\pi_0 = 0, \pi_v \in \mathbb{R}, \quad v \in V, \quad (3.17)$$

$$X_t - X_{t-1} \leq M y_t, \quad 2 \leq t \leq T, \quad (3.18)$$

$$X_1 \leq M y_1 \quad (3.19)$$

$$y_t \in \{0, 1\}, \quad 1 \leq t \leq T, \quad (3.20)$$

$$X \in \mathbb{X} \quad (3.21)$$

FIGURE 4 – Reformulation du problème Maître. [13]

Les Inégalités (3.12) à (3.15) modélisent le calcul du plus long chemin au sein du graphe partiel $\mathcal{G}'$. Elles imposent que la différence de potentiel $\pi_{t,j} - \pi_{(t-1),i}$ entre deux nœuds successifs connectés par un arc $a_{t,i,j} \in A'$ soit au moins égale au coût de cette transition. Comme détaillé dans la définition du graphe de budget (voir Définition A.1, située en Annexe A), ce coût correspond à la contribution maximale pour la fonction

objectif. Le modèle évalue simultanément la pénalité d'une situation de sur-stockage (c^I) et d'une situation de rupture (c^B). Ces équations intègrent également le coût fixe de lancement c^P et, pour les arcs arrivant à la période T , la déduction des revenus générés par le prix de vente b^P . L'Équation (3.17) initialise le potentiel du nœud source π_0 à 0. Le dernier bloc de contraintes reprend celles de la chaîne de production, telles qu'elles ont été introduites dans le modèle de la Figure 4. Nous y retrouvons les contraintes d'activation de type *Big-M* ((3.18) et (3.19)). Les Équations (3.20) et (3.21) garantissent le respect des domaines de définition et des capacités globales de l'usine, définies par le polytope $\mathcal{X}$.

3.3.2 Le sous-problème Adv

Une fois le problème Maître résolu pour une itération donnée, il fournit un plan de production X . L'objectif du sous-problème adverse est alors d'évaluer la robustesse de cette solution en cherchant non pas un unique pire scénario, mais un ensemble de scénarios défavorables qui viendront accroître les connaissances du problème Maître. Cet ensemble prend la forme d'un nouveau graphe partiel de budget, noté $\mathcal{G}''$.

Pour construire ce graphe, la méthode s'appuie sur un algorithme de programmation dynamique (voir l'Algorithme 4, situé en Annexe C). La résolution mathématique de l'algorithme repose sur le principe d'optimalité de Bellman [1]. Ce principe fondamental énonce qu'une politique optimale possède la propriété suivante : « quels que soient l'état et la décision initiaux, les décisions ultérieures doivent constituer une politique optimale par rapport à l'état résultant de la première décision ». Dans notre modèle, l'état du système est défini à chaque étape par le couple (t, j) , où t représente la période et j le budget d'incertitude cumulé consommé à cet instant. L'équation de Bellman permet de calculer le coût du plus long chemin $\pi[t][j]$ de manière récursive :

$$\pi[t][j] = \max_{\substack{i \leq j \\ (i \rightarrow j) \text{ valide}}} (\pi[t-1][i] + c[t][i][j]) \quad (3.22)$$

Dans cette équation, i représente le budget consommé à la période précédente $t-1$, et $c[t][i][j]$ désigne le coût local de la transition. Dans un premier temps, un calcul de plus long chemin est réalisé sur le graphe de budget complet $\mathcal{G}$, afin de déterminer le coût maximal pour atteindre chaque nœud depuis l'origine v_0 . Cette étape s'effectue rapidement du fait que le graphe est orienté et acyclique. Ensuite, une phase de remontée (*backtracking*) est réalisée depuis le nœud final v_{T+1} . À chaque étape du *backtracking*, au lieu de ne conserver que l'arc menant au pire chemin absolu, l'algorithme sélectionne un sous-ensemble d'arcs appartenant aux chemins critiques.

C'est cette sélection d'arcs qui définit la structure et la taille du graphe partiel $\mathcal{G}''$. Pour être plus précis, un critère de relaxation ($\tau \in [0, 1]$) a été mis en place sur la définition des pires chemins. Lors de la première phase, le coût du plus long chemin, noté C (ou *ub_cost* dans les algorithmes), sert de référence pour définir la notion de chemin critique. Un scénario est donc considéré comme critique si sa longueur est

supérieure ou égale à une borne relaxée C_{relax} , définie dans l'Équation (3.23).

$$C_{relax} = C - (1 - \tau)|C| \quad (3.23)$$

Si $\tau = 1$, alors $C_{relax} = C$; seuls les chemins ayant le même coût que celui du pire chemin absolu sont considérés comme critiques et ajoutés au sous-graphe. À l'inverse, si $\tau = 0$, l'ensemble des chemins sous-optimaux est exploré. Nous nous attendons à ce que la diminution de τ entraîne une augmentation du temps de résolution, mais permette une potentielle baisse du nombre d'itérations globales.

3.3.3 L'opération de fusion

Dans l'approche classique de Benders Adverse, l'ajout d'un nouveau scénario se traduit par une simple concaténation dans une liste. Dans le cadre de la BAE, il est nécessaire de combiner le graphe partiel de l'itération courante ($\mathcal{G}'$) avec celui généré par le sous-problème adverse ($\mathcal{G}''$). Leur fusion a donc été définie par l'Opération (3.24).

$$\text{Merge}(\mathcal{G}', \mathcal{G}'') = (V, A' \cup A'') \quad (3.24)$$

où A' et A'' sont les ensembles contenant les arcs actifs des graphes des itérations $\mathcal{G}'$ et $\mathcal{G}''$.

Toutefois, cette opération de fusion soulève une limite sur laquelle j'ai travaillé. Elle concerne l'aspect combinatoire de cette méthode [13]. En effet, la fusion de deux graphes partiels peut générer de nouveaux chemins qui n'existaient dans aucun des deux graphes initiaux. Ainsi, l'ensemble des scénarios du graphe fusionné est potentiellement plus grand que la somme des scénarios des graphes de départ. Il devient donc nécessaire de travailler sur l'affinage des coupes combinatoires afin de ne pas perdre l'avantage lié à cette extension.

3.3.4 Stratégies de sélection des pires scénarios

Comme évoqué précédemment, la taille et la densité du graphe partiel de budget $\mathcal{G}''$ dépendent entièrement de la politique de sélection des arcs lors de la phase de *backtracking*. Dans la littérature, deux heuristiques principales ont été proposées pour effectuer ce tri parmi les chemins critiques :

- *SelectAll* : Cette approche exhaustive consiste à sélectionner tous les arcs entrants dont les nœuds d'origine appartiennent à l'un des plus longs chemins dans le graphe partiel. Autrement dit, le graphe généré contient l'intégralité des pires scénarios pour la solution courante. Bien qu'elle fournisse une information très complète au problème Maître, cette méthode tend à faire exploser très rapidement le nombre d'arcs après quelques opérations de fusion. Le graphe partiel devient alors trop dense et difficile à exploiter.
- *SelectFew* : Pour faire face à ce problème, cette méthode propose de restreindre la sélection. Lors du parcours, un premier arc critique est systématiquement

choisi, puis un autre arc critique potentiel n'est ajouté qu'avec une certaine probabilité, suivant une loi de Bernoulli de paramètre p_{Few} . Cette restriction probabiliste permet de garder un graphe partiel de taille réduite, car il existe un grand nombre de pires scénarios équivalents.

Ces deux méthodes illustrent le compromis soutenu par les auteurs de *BAE*. Il s'agit de fournir suffisamment de scénarios critiques au Maître pour le guider efficacement, tout en contrôlant la taille du graphe afin de garantir des temps de résolution acceptables. Cette nécessité de filtrage met en évidence l'intérêt de développer des heuristiques de sélection plus structurelles. Au lieu de s'en remettre à une sélection probabiliste ou exhaustive, il devient pertinent de chercher à extraire des chemins non seulement critiques, mais aussi les plus diversifiés possibles. Le but est de maximiser la quantité d'information transmise tout en minimisant la redondance des scénarios. C'est ce point que nous aborderons dans la prochaine section.

3.3.5 L'algorithme de Benders Adverse Étendue

En combinant les éléments précédents, nous avons le schéma itératif de la méthode de Benders Adverse Étendue. De la même manière que nous initialisons l'algorithme de Benders Adverse (voir l'Algorithme 1) par le scénario nominal, ici, nous initialisons BAE par le graphe de budget $\hat{\mathcal{G}}$ ne contenant que le scénario nominal.

Algorithm 2: Extended Adversarial Benders Method

```

1  $\mathcal{G}' \leftarrow \hat{\mathcal{G}}$ 
2  $(X, v) \leftarrow \text{SolveMaster}(U)$ 
3  $(\mathcal{G}'', v') \leftarrow \text{SolveAdversarial}(X)$ 
4 while  $v \neq v'$ 
5    $v \leftarrow v'$ 
6    $\mathcal{G}' \leftarrow \text{Merge}(\mathcal{G}', \mathcal{G}'')$ 
7    $(X, v) \leftarrow \text{SolveMaster}(U)$ 
8    $(\mathcal{G}'', v') \leftarrow \text{SolveAdversarial}(X)$ 
9 return  $(X, v)$ 

```

4 Objectifs

Cette section détaille les objectifs scientifiques de cette recherche. Pour cela, nous nous appuyerons sur une première analyse des performances de référence.

4.1 Manipulation théorique et restructuration du modèle

La première phase du stage a été consacrée à l’assimilation des concepts d’optimisation robuste appliqués à la problématique du *Lot-Sizing* sous incertitude. Afin de conceptualiser le fonctionnement de la méthode de Benders Adverse, l’algorithme a d’abord été déroulé, puis appliqué à des instances de taille réduite (le détail de ces itérations est consultable en Annexe M).

Au-delà de l’appropriation théorique, une partie de ce travail a reposé sur de l’implémentation logicielle. En effet, après avoir récupéré une version du code en C++ utilisée dans la thèse de [13], il a été nécessaire de le reconstruire de manière stable. Ce travail de refonte a exigé de modéliser et d’implémenter plusieurs contraintes mathématiques manquantes par rapport à la formulation du problème de *Lot-Sizing* susmentionné (voir la modélisation du Problème 3.2). La liaison avec le solveur exact CPLEX a également dû être reconstruite pour garantir la viabilité des expérimentations futures.

Tout d’abord, nous avons dû régler des erreurs de conception lors de l’initialisation des instances. La logique de traitement est détaillée dans l’Algorithme 5, situé en Annexe D. Il a été nécessaire de repenser la définition des bornes d’incertitude Δ_t . Elles correspondent à l’incertitude que nous pouvons allouer localement à chaque instant $t \leq T$. En effet, une affectation aléatoire de ces bornes peut générer des instances incohérentes (ex. allouer un budget d’incertitude élevé à une période de faible demande). Pour pallier ce problème, le budget d’incertitude global Γ est maintenant modulé par une marge de manœuvre adverse. Il est, par ailleurs, distribué au prorata de la demande marginale de chaque période par rapport à la demande totale de l’horizon. La capacité de perturbation de l’adversaire est désormais concentrée sur les périodes à forts enjeux (resp. à forte demande). L’incertitude locale ne peut pas dépasser la demande nominale de la période.

Concernant les problèmes d’implémentation logicielle, nous pouvons citer quelques exemples. Tout d’abord, lors de la phase de *backtracking*, pour reconstruire le pire scénario d’attaque, le code utilisait une égalité stricte pour retrouver le chemin optimal. Face à des valeurs fractionnaires calculées par CPLEX, l’égalité stricte échouait à cause des limites de précision des flottants. Nous avons donc reformulé les contraintes en introduisant une tolérance ε . Il y avait également une erreur d’indexation dans les boucles de génération des contraintes du Maître. Les différents nœuds de la période T n’étaient pas reliés au puits à $T + 1$. Cette rupture de chemin faussait le calcul des coûts duaux à la dernière période, bloquant l’objectif à une valeur sous-optimale.

Les problèmes les plus complexes à résoudre provenaient de l’alignement entre la théorie mathématique du problème et les contraintes physiques. Le MILP autorisait

des solutions mathématiquement valides, mais physiquement impossibles ou inversement. Une première contrainte, héritée du code initial, forçait la production du Maître à ne jamais dépasser le plan nominal. Cette contrainte détruisait le principe d'optimisation robuste, qui doit permettre de constituer un stock de sécurité. Nous l'avons supprimée et en avons ajouté une, impliquant que la production cumulée ne peut pas décroître ($X_t \geq X_{t-1}$). Un deuxième problème a été soulevé. Il concernait la possibilité d'avoir des ventes négatives. Une attaque de l'adversaire à la baisse pouvait faire chuter la courbe de la demande cumulée. L'équation de flux du Maître, qui égalisait la somme des ventes passées à la demande nette, forçait donc le solveur à annuler des ventes déjà effectuées. Nous avons alors ajouté des contraintes en conséquence (voir les différents algorithmes présentés en Annexe).

Initialement, les instances utilisées étaient récupérées via la plateforme *CSPLib*. Pour plus de flexibilité et afin de modéliser les instances en suivant le problème de *Lot-Sizing*, nous avons développé un script en *Python*. Il produit donc des fichiers *txt* contenant l'instance générée. De plus, tous les différents scripts permettant d'étudier les résultats des simulations ont été créés sur mesure en *Python* et *R* (voir l'Annexe N pour plus d'explications).

Nous ne remettons pas en cause la viabilité des résultats contenus dans [13], car il ne s'agit certainement pas de la dernière version utilisée.

4.2 Analyse des performances de référence

Une fois l'environnement de développement opérationnel, l'objectif suivant a été de reproduire les expérimentations de référence pour évaluer le comportement des différentes approches existantes. L'idée défendue par la méthode *BAE* est que l'extraction d'un graphe de pires scénarios permet de réduire le nombre d'itérations afin de gagner en temps de calcul. Pour valider cette première hypothèse, nous avons mené une batterie de tests sur des instances générées aléatoirement. Ces instances simulent une planification sur $T = 52$ périodes (représentant une année de production) pour 20 références de produits distincts. Une périodicité trimestrielle de 13 semaines a été mise en place. La probabilité qu'une référence j soit demandée à un instant t suit une loi de Bernoulli de paramètre $p_{demand} = 0,7$ lors des périodes denses et $(1 - p_{demand})$ lors des périodes creuses. L'étude a été réalisée en faisant varier le budget d'incertitude Γ dans son ensemble de définition $\mathcal{D}_\Gamma = \{1, 11, \dots, 91, 101\}$ et $\tau = 1$. En ce qui concerne l'heuristique de sélection partielle BAE_{Few} , la décision de conserver un arc alternatif lors du *backtracking* suit également une loi de Bernoulli de paramètre⁵ $p_{Few} = \frac{1}{T}$. Le Tableau 1 synthétise ces résultats de référence en mesurant le nombre moyen d'itérations et le temps moyen de résolution en secondes pour les couples de la forme $(\Gamma_i, \{BA\})$ et $(\Gamma_i, \tau, \{BAE_{All}, BAE_{Few}\})$, $\forall \Gamma_i \in \mathcal{D}_\Gamma$.

L'analyse de ces résultats préliminaires met en évidence les limites des approches actuelles. Tout d'abord, nous pouvons noter que BAE_{All} est de loin la moins perfor-

5. Nous avons repris l'ordre de grandeur en $1/T$ proposé par [13].

Γ	Method	BA		BAE_{All}		BAE_{Few}	
		time	iter.	time	iter.	time	iter.
1		0,015	2,96	0,004	1,28	0,004	1,28
21		0,208	8,28	0,093	1,0	0,073	1,0
41		0,292	8,40	0,299	1,0	0,271	1,0
61		0,422	8,24	0,689	1,0	0,507	1,0
81		0,610	8,28	1,390	1,0	1,057	1,0
101		0,833	8,48	1,833	1,0	1,789	1,0

TABLE 1 – Extrait du temps moyen et du nombre moyen d’itérations entre BA , BAE_{All} & BAE_{Few} pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$. Les valeurs sont arrondies au millièrme.

mante en termes de temps moyen de calcul. Cela met en avant la nécessité d’élaguer plus efficacement l’arbre de décision. Si la méthode de compilation de connaissances BAE_{Few} domine l’approche classique BA en termes de temps de calcul et d’itérations pour les instances à faible incertitude, elle n’est pas toujours la plus performante. Par conséquent, le filtrage aléatoire n’est pas une stratégie soutenable à grande échelle.

4.3 Définition de l’objectif principal

Les travaux de [14] sur la compilation de connaissances ont démontré que la factorisation permet d’accélérer la résolution du problème Maître en fusionnant les tronçons communs des scénarios adverses. Cependant, comme nous venons de le voir, une extraction exhaustive sature rapidement le graphe partiel de budget. Cela mène à une explosion combinatoire dès lors que le budget d’incertitude Γ augmente. L’enjeu central de cette étude est donc de concevoir des heuristiques d’extraction capables de limiter strictement le nombre de scénarios renvoyés au Maître à un quota N , tout en maximisant la qualité de l’information contenue dans ce sous-ensemble.

5 Propositions

Cette section détaille l'évolution algorithmique de nos heuristiques. Nous présentons tout d'abord des méthodes de référence extrayant un ou deux chemins spécifiques (*KCU*, *KCUD*) ainsi qu'une approche stochastique (*KCRDKL*). Celle-ci nous servira de *baseline* pour évaluer nos propositions majeures basées sur l'orthogonalité spatiale (*KCOG*, *KCOGL*). Pour plus de lisibilité, la majorité des algorithmes présentés se situent en annexe.

Il est important de définir deux ensembles que nous manipulerons tout au long du rapport : l'ensemble optimal P_{opt} qui contient tous les scénarios optimaux ; et l'ensemble relaxé P_{cand} qui contient les scénarios sous-optimaux. L'ensemble relaxé est construit par rapport au coefficient de relaxation $\tau \in [0, 1]$ et au pire coût optimal C (ou *ub_cost* dans le code). Formellement, un scénario est considéré comme étant sous-optimal si son coût appartient à l'ensemble $[C_{relax}, C[$, où C_{relax} est défini par l'Équation (3.23). Il est alors évident que si $\tau = 0$, tous les scénarios du graphe sont considérés. À l'inverse, $\tau = 1$ considère uniquement les scénarios optimaux. Nous pouvons conjecturer dès lors que l'évolution de τ aura un impact important sur les performances. Notons que seules *KCOG*, *KCOGL*, *KCRDK* et *KCRDKL* utilisent ce paramètre de relaxation.

Cette notion d'ensemble relaxé va de pair avec l'Algorithme 9 (voir Section 5.3.2) de recherche en profondeur (DFS). En effet, il est nécessaire de différencier les deux types d'algorithmes que nous utiliserons. Lors du *backtracking*, les heuristiques *BAE*, *KCU* et *KCUD* utilisent une extraction gloutonne. Cette méthode itérative sélectionne le premier arc optimal rencontré à chaque pas de temps et s'y engage définitivement. Elle garantit l'extraction très rapide d'une (ou deux) trajectoire(s) unique(s). À l'inverse, le DFS, utilisé par *KCOG*, *KCOGL*, *KCRDK* et *KCRDKL*, explore systématiquement les différentes bifurcations de l'arbre de décisions. Elle est donc plus lourde en temps de calcul.

Préambule et notations. Afin d'alléger les descriptions algorithmiques à venir, nous définissons les notations globales suivantes, communes à toutes nos heuristiques :

- $\pi_{t,j}$: Coût maximal cumulé depuis l'origine ($t = 0$) jusqu'à la fin de la période t , en ayant consommé exactement j unités de budget d'incertitude.
- c : Matrice des coûts de transition d'un état à un autre.
- Δ_t et d_t : Respectivement, la borne locale d'incertitude et la demande nominale à une période donnée.
- *ub_cost* : Le pire coût adverse absolu renvoyé par la programmation dynamique.
- $\tau \in [0, 1]$: Coefficient de relaxation.
- *Calculate_Bound*(τ, ub_cost) : Fonction qui permet de définir la borne de l'ensemble relaxé via l'Équation (3.23).
- N : Quota strict de scénarios que le sous-problème doit renvoyer au Maître à chaque itération.

- L : Limite de troncature pour l’exploration du graphe lors de la recherche en profondeur (DFS).
- ε : Tolérance numérique (fixée à 10^{-4}) utilisée pour comparer les coûts flottants et valider l’équation de Bellman

5.1 L’heuristique *KCU* (*KC with a unique optimal path*)

5.1.1 Introduction

Contrairement à l’approche *BAE* qui inonde le Maître d’un sous-graphe dense représentant de multiples scénarios, *KCU* limite strictement l’extraction à un unique scénario optimal par itération. *KCU* utilise le même problème Maître que les méthodes *KC* décrites dans ce document; seul l’algorithme d’extraction du sous-problème diffère. Il est important de souligner que cette méthode cible exclusivement les chemins strictement optimaux.

5.1.2 Extraction et factorisation

Pour l’extraction de ce chemin unique, l’utilisation d’un algorithme de recherche en profondeur (DFS) est superflue. Nous implémentons une approche par remontée gloutonne, détaillée dans l’Algorithme 6 (situé en Annexe E). Voici l’idée générale de l’heuristique :

- **Identification de la racine (l. 3 à 8)** : L’algorithme parcourt les nœuds terminaux et sélectionne le premier budget γ vérifiant $|\pi_{T,\gamma} - ub_cost| < \varepsilon$. La recherche s’arrête dès ce nœud trouvé.
- **Remontée gloutonne (l. 11 et suivantes)** : À chaque pas de temps suivant, l’algorithme évalue les antécédents possibles. Dès qu’un arc respectant la condition d’optimalité de Bellman est identifié, il est validé et ajouté au sous-graphe.
- **Déterminisme** : L’ordre séquentiel d’évaluation (budgets croissants, puis type 0, puis type 1)⁶ rend cette extraction strictement déterministe. En cas de dégénérescence, c’est-à-dire lorsque plusieurs antécédents $(t-1, i)$ conduisent au même potentiel optimal $\pi_{t,i}$, l’algorithme sélectionne systématiquement le premier rencontré selon cet ordre lexicographique. Ce mécanisme assure la reproductibilité de l’extraction. Cependant, il est important de noter que *KCU* ne garantit pas l’exhaustivité des chemins optimaux. Elle extrait un unique chemin optimal parmi l’ensemble des solutions de coût minimal. Cette limitation est compensée par sa faible complexité (voir Section 5.1.3).

5.1.3 Algorithme glouton

L’Algorithme 6 a une complexité en $\mathcal{O}(T \cdot \Gamma)$. Le *backtracking* s’effectue sur T périodes. À chaque période, nous explorons au pire l’ensemble des budgets antérieurs possibles,

6. Le problème a été modélisé en séparant les deux types de coût. Nous avons le type 0 qui modélise le choix d’attaquer en surstockage et le type 1 en rupture.

limité par Γ . Le temps de vérification des transitions est en $\mathcal{O}(1)$.

5.1.4 Limites de l'approche *KCU*

Bien que cette méthode allège l'extraction en évitant le parcours en profondeur, elle ne garantit aucune diversité spatiale. Si l'espace d'incertitude contient de multiples chemins optimaux mathématiquement équivalents, mais topologiquement très distants, *KCU* n'exploitera toujours que le même profil d'attaque, ce qui peut ralentir la convergence globale du Maître.

5.2 L'heuristique *KCUD* (*KC with two distant optimal paths*)

5.2.1 Introduction

Cette heuristique s'appuie essentiellement sur *KCU*. Par construction, l'heuristique *KCU* extrait le scénario qui retarde au maximum la consommation de l'incertitude. Il s'agit d'un choix de programmation; nous aurions tout aussi bien pu retourner le scénario dans lequel l'incertitude a été utilisée en totalité le plus tôt possible. Ce choix nous a conduits à nous interroger sur ce qu'il en serait si nous remontions ces deux types de scénarios à chaque itération. Cela correspondrait à une approche « d'orthogonalité », car les deux chemins retournés seraient les scénarios optimaux les plus distants en termes de temporalité d'incertitude. Nous nous attendons à ce que le temps total de *KCUD* puisse être inférieur à celui de *KCU* et qu'il y ait moins d'itérations. L'algorithme reprend l'architecture de *KCU*; il s'agit d'exploiter la structure de la matrice de programmation dynamique pour extraire non pas un, mais deux chemins distincts. Ce premier chemin est obtenu en privilégiant systématiquement les transitions issues des budgets les plus faibles (consommation tardive), tandis que le second privilégie les transitions issues des budgets élevés (consommation précoce) (voir la Figure 5).

Cependant, bien que *KCUD* soit prometteuse, il faut rappeler un point important : la dégénérescence vers un chemin unique. En effet, il est tout à fait possible que la matrice de programmation dynamique ne contienne qu'un seul scénario optimal. Dans ce cas, les itérations ascendantes et descendantes renverront exactement le même scénario. L'effet d'orthogonalité de *KCUD* est alors nul; cette méthode serait équivalente à la méthode *KCU*.

5.2.2 Algorithme glouton

L'Algorithme 7 (situé en Annexe F) a une complexité en $\mathcal{O}(T \cdot \Gamma)$. La recherche des extrema initiaux à $t = T$ prend $\mathcal{O}(\Gamma)$. Ensuite, l'algorithme fait appel à deux extractions gloutonnes (voir l'Algorithme 8 situé en Annexe F), chacune s'exécutant en $\mathcal{O}(T \cdot \Gamma)$.

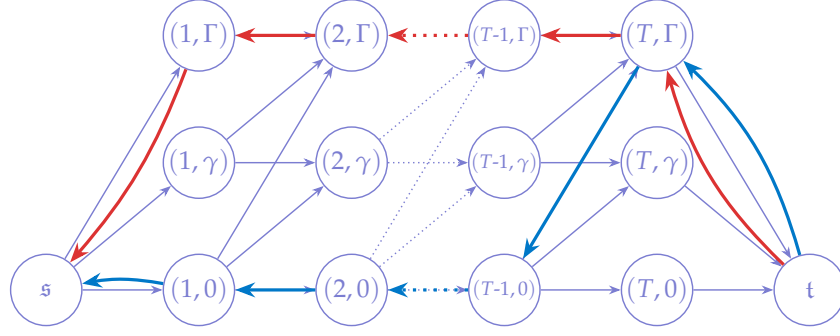


FIGURE 5 – Sélection des deux scénarios optimaux les plus distants temporellement lors de la phase **Adv** de *backtracking*.

5.2.3 Limites de l'approche *KCUD*

Intuitivement, l'heuristique *KCUD* vise à maximiser l'écart entre les scénarios extraits en forçant la consommation de l'incertitude aux deux extrêmes temporels. Il serait alors tentant de considérer *KCUD* comme un cas particulier de *KCOGL* ($N = 2$), où N est le quota de scénarios à remonter (cette heuristique sera définie dans la Section 5.3). Or, d'un point de vue mathématique, ces deux méthodes diffèrent. *KCUD* garantit l'écartement temporel, mais pas l'orthogonalité.

Comme indiqué précédemment, *KCUD* ne garantit pas le renvoi de deux scénarios optimaux distincts. Au vu des deux limites évoquées, nous pourrions considérer l'approche *KCOGL* comme une solution. Par sa construction, elle évite les goulots d'étranglement et permet également d'éviter le retour de deux mêmes scénarios en prenant, si besoin, des scénarios dans l'ensemble relaxé.

5.3 L'heuristique *KCOG* (KC with orthogonality)

5.3.1 Introduction

L'objectif de *KCOG* est d'extraire un sous-ensemble de scénarios adverses qui soient à la fois très pénalisants et spatialement diversifiés. L'idée sous-jacente est qu'en remontant des scénarios très diversifiés, nous pourrions attaquer le Maître sous des angles différents et ainsi gagner en performance.

Cette heuristique s'articule autour de deux composants algorithmiques appelés dans le sous-problème. Une première phase d'extraction (voir Section 5.3.2) qui s'appuie sur un algorithme de recherche en profondeur déterministe pour parcourir le graphe des états de la programmation dynamique et remonter les chemins critiques. Ainsi qu'une seconde phase de filtrage et de sélection (voir Sections 5.3.3 et 5.3.4) qui s'appuie sur le calcul de la distance de Manhattan pour identifier un sous-ensemble de chemins maximisant la dispersion spatiale par rapport à l'espace d'incertitude (approche **MaxMin**).

L'architecture de *KCOG* contourne une sélection naïve en structurant l'extraction autour de trois phases majeures :

1. **Génération des bassins de candidats** : À l'aide d'une recherche en profondeur, l'algorithme extrait les pires scénarios absolus (P_{opt}) ainsi que les scénarios sous-optimaux (P_{cand}).
2. **Garantie de convergence** : Le premier chemin critique absolu identifié ($P_{opt}[0]$) est systématiquement sélectionné afin d'assurer l'amélioration stricte de la borne du problème Maître.
3. **Diversification spatiale** : Pour atteindre le quota de N scénarios, l'algorithme filtre l'ensemble relaxé en appliquant le critère **MaxMin** basé sur la distance de Manhattan. Il sélectionne itérativement les attaques adverses les plus orthogonales à celles déjà retenues.

5.3.2 Algorithme de recherche en profondeur (DFS)

Bien que le coût maximal adverse à l'horizon T soit connu, les séquences de décisions qui y mènent doivent être reconstruites. Le DFS procède donc à un *backtracking* déterministe depuis les nœuds terminaux jusqu'à l'origine du graphe. La mécanique de *backtracking* (voir la Figure 17 en Annexe H) repose sur les principes suivants :

- **Exploration des antécédents (l. 9)** : Depuis un état courant (t, j) , l'algorithme évalue tous les budgets i ⁷ potentiellement consommés à la période précédente $(t - 1)$ ⁸.
- **Faisabilité des transitions (l. 10 & 11)** : L'algorithme s'assure que la variation de budget $(j - i)$ respecte le budget d'incertitude local Δt_{t-1} . De plus, dans le cas d'un sur-stockage (arc de type 0), l'attaque de l'adversaire réduisant la demande nominale ne doit pas conduire à une demande effective négative, cela est imposé par $(j - i) \leq sol.dt[t - 1]$.
- **Troncature (l. 1)** : Le parcours étant réalisé sur un DAG combinatoire, le nombre de pires scénarios peut exploser. Une limite L est imposée pour stopper l'extraction. Par conséquent, bien que l'algorithme soit purement déterministe et sans doublon, il ne garantit pas l'exhaustivité de l'extraction si l'espace des chemins optimaux dépasse L .

L'Algorithme 9 (situé en Annexe G) a une complexité en $\mathcal{O}(L \cdot T \cdot \Gamma)$. Il s'agit d'un parcours en profondeur sur un DAG. La troncature limite le nombre de feuilles explorées à L . Chaque chemin ayant une longueur maximale T , et l'exploration des budgets i étant bornée par Γ , la complexité est restreinte par la limite L .

5.3.3 Algorithme de calcul de distance

Pour quantifier la divergence entre deux scénarios adverses, différentes métriques ont été considérées. La première a été la distance de Jaccard, définie comme le complémentaire de son indice de similarité de la façon suivante.

7. Un compromis algorithmique a été fait lors de l'évaluation des nœuds à une période t . Nous partons du nœud le plus bas dans le graphe de budget.

8. La nature cumulative du budget impose que $0 \leq i \leq j$.

$$D_J(A, B) = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} = \frac{d_H(A, B)}{|A \cup B|} \quad (5.1)$$

où d_H est la distance de Hamming⁹.

Cette notion de distance s'appuie essentiellement sur la différence de chemin emprunté par les deux scénarios. La dimension d'incertitude n'était pas prise en compte ; nous avons finalement opté pour une toute autre mesure, la distance de Manhattan associée à la norme L_1 . Introduite par Hermann Minkowski [10], elle s'est depuis imposée comme une distance utilisée naturellement dans plusieurs domaines. Nous pouvons la trouver notamment en optimisation combinatoire, dans des algorithmes comme Dijkstra ou A^* , grâce à sa propriété d'inégalité triangulaire qui permet de réduire les espaces de recherche. Outre ses propriétés supplémentaires de non-négativité, d'identité des indiscernables et de symétrie, elle est adaptée car elle permet de pénaliser linéairement les déviations d'incertitude [4].

Formellement, pour un espace vectoriel sur $\mathbb{R}^n$, la norme L_1 d'un vecteur $\vec{x}$ est définie par $\|\vec{x}\|_1 = \sum_{i=1}^n |x_i|$.

Dans notre modèle, l'idée n'est plus d'évaluer par où passe le chemin, mais d'isoler l'énergie dépensée par l'adversaire à chaque pas de temps. Soient A et B deux scénarios de planification sur un horizon T . À une période $1 \leq t \leq T$ donnée, la déviation locale d'un scénario correspond à la différence entre le budget cumulé d'arrivée j et le budget cumulé de départ i . On note ainsi les déviations marginales respectives : $\delta_t^A = A_{t,j} - A_{t,i}$ et $\delta_t^B = B_{t,j} - B_{t,i}$ avec $\delta_t^A, \delta_t^B \in \mathbb{N}$.

La divergence locale entre les scénarios A et B , à l'instant t , est évaluée par la différence absolue $|\delta_t^A - \delta_t^B|$. La distance totale $d_{L1}(A, B)$ est alors donnée par la Formule (5.2), telle qu'implémentée dans l'Algorithme 3 (voir la Figure 6).

$$d_{L1}(A, B) = \sum_{t=1}^T |\delta_t^A - \delta_t^B| \quad (5.2)$$

La complexité de l'Algorithme 3 est linéaire en $\mathcal{O}(T)$, puisqu'il nécessite un unique parcours des séquences de décisions.

5.3.4 Algorithme de sélection MaxMin

Une fois la matrice des chemins candidats générée par le DFS, nous implémentons un algorithme de dispersion, de type **MaxMin**, pour sélectionner un sous-ensemble restreint de N scénarios. Contrairement au problème global qui est un **MinMax** sur les coûts, l'objectif est de maximiser la distance minimale entre les scénarios retenus (voir

9. La distance de Hamming (L_0) est définie dans la littérature (voir [9]) comme une métrique de type « arc overlap », c'est-à-dire que la distance est définie comme étant la différence d'arcs empruntés entre les deux chemins. Formellement, il s'agit d'une distance L_1 restreinte aux vecteurs binaires $x \in \{0, 1\}^{|E|}$:

$$d_H(x_1, x_2) = |x_1 \Delta x_2| = |x_1 \cup x_2| - |x_1 \cap x_2|.$$

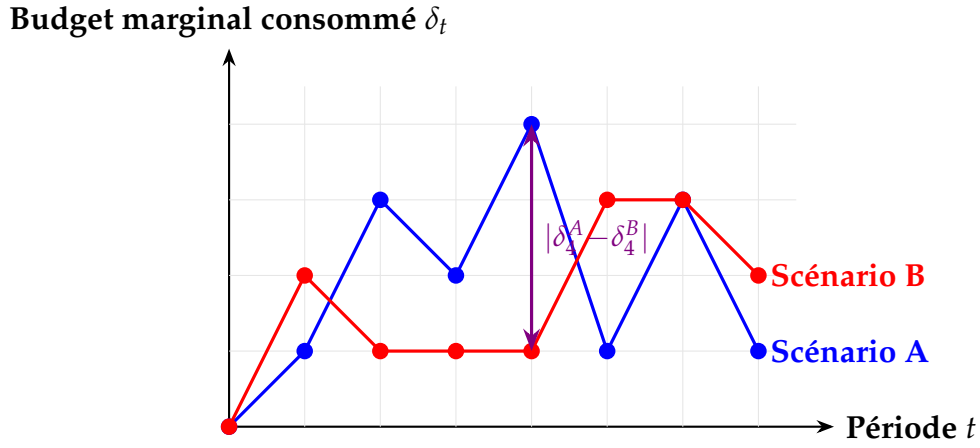


FIGURE 6 – Définition de la métrique de distance.

Algorithm 3: Calculate Manhattan Distance (*KCOG*, *KCOGL*)

Require: Path A , Path B
Ensure: Distance $d_{L1} \geq 0$

```

1 if  $|A| \neq |B|$ 
2   return 0
3  $sum\_diff \leftarrow 0$ 
4 for  $k = 1$  to  $|A|$ 
5    $\Delta_A \leftarrow A[k].j - A[k].i$  // Marginal budget consumed by A at  $k$ 
6    $\Delta_B \leftarrow B[k].j - B[k].i$ 
7    $sum\_diff \leftarrow sum\_diff + |\Delta_A - \Delta_B|$ 
8 return  $sum\_diff$ 

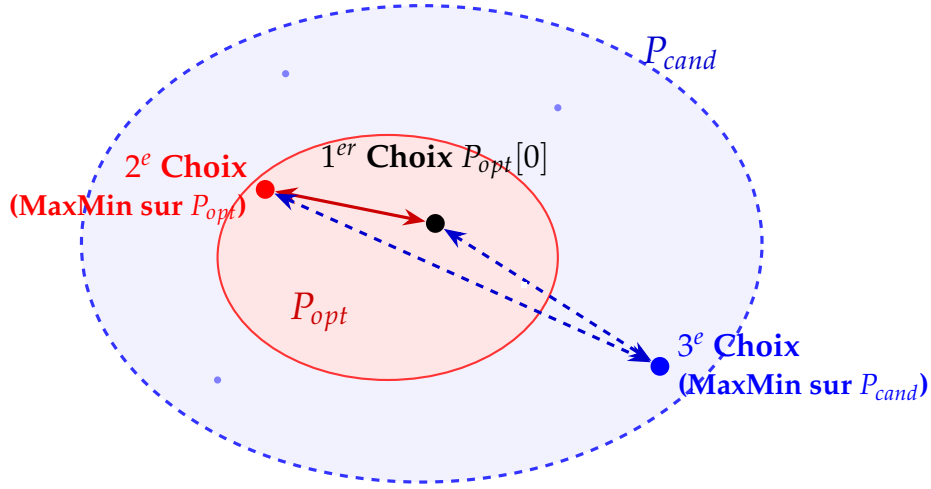
```

la Figure 7). Ce sous-ensemble sera agrégé dans le sous-graphe renvoyé au problème Maître. La procédure est itérative :

- **Évaluation de la distance (Boucle Min, l. 1) :** Pour chaque chemin candidat de l'ensemble relaxé P_{cand} , nous calculons sa distance par rapport à l'ensemble des chemins déjà sélectionnés (P_{sel}). La plus petite de ces distances (min_dist) définit son degré d'isolement : une valeur faible indique une forte redondance avec un scénario déjà connu du Maître.
- **Sélection du candidat (Boucle Max, l. 11) :** L'algorithme retient le candidat possédant le degré d'isolement min_dist le plus élevé. Ce scénario, considéré comme le plus orthogonal à la base de connaissances actuelle, est concaténé dans l'ensemble P_{sel} .

La complexité de l'Algorithme 10 (situé en Annexe I) est en $\mathcal{O}(L \cdot N^2 \cdot T)$ ¹⁰. La boucle *while* principale tourne N fois. À chaque itération, nous parcourons les L candidats de l'ensemble relaxé. Pour chaque candidat, nous calculons la distance par rapport aux scénarios déjà sélectionnés (au pire N), ce qui coûte $\mathcal{O}(T)$. Total : $\sum_{s=1}^N (L \times s \times T) = \mathcal{O}(L \cdot N^2 \cdot T)$.

10. Il est possible de réduire sa complexité à $\mathcal{O}(L \cdot N \cdot T)$ en mémorisant les distances.

FIGURE 7 – Schéma de fonctionnement de la discrétisation **MaxMin**.

5.3.5 Limites de l'approche *KCOG*

Alors que cette méthode garantit l'inclusion d'une attaque optimale, la diversification spatiale s'applique aveuglément sur l'ensemble relaxé. Par conséquent, l'heuristique accepte le compromis de potentiellement écarter un scénario optimal peu distant, au profit d'un scénario sous-optimal très orthogonal. C'est pour pallier ce compromis qu'une évolution lexicographique sera introduite à la Section 5.4.

5.4 L'heuristique *KCOGL* (*KCOG* lexicographical)

5.4.1 Introduction

L'heuristique *KCOGL* est une évolution lexicographique de *KCOG*. Elle corrige la limite évoquée précédemment en forçant la sélection **MaxMin** en priorité sur l'ensemble des scénarios optimaux P_{opt} , avant d'exploiter l'ensemble relaxé P_{cand} .

5.4.2 Algorithme d'extraction aléatoire priorisé

L'Algorithme 11 (situé en Annexe J) a la même complexité que ceux de la Section 5.3. Elle est en $\mathcal{O}(L \cdot T \cdot \Gamma + L \cdot N^2 \cdot T)$ car elle combine la complexité du DFS ainsi que celle de l'Algorithme 10 pour le filtrage spécial.

5.4.3 Limites de l'approche *KCOGL*

L'heuristique *KCOGL* représente l'application finale de notre stratégie de sélection spatiale. Toutefois, l'approche **MaxMin** impose une limite pratique. Nous anticipons que *KCOGL* surpasse la *baseline* *KCRDKL* (voir Section 5.5) lorsque N reste limité. À l'inverse, pour des valeurs de N élevées, le surcoût computationnel du calcul des distances risque de s'avérer supérieur au gain d'itérations. Cela rendra l'approche stochastique plus performante. Enfin, nous pouvons noter que, structurellement, l'exé-

cution de $KCOGL(N = 1, L = 1)$ équivaut à l'approche KCU détaillée à la Section 5.1.

5.5 L'heuristique $KCRDKL$ (KC random K lexicographical)

5.5.1 Introduction

La structure algorithmique de $KCRDKL$ est identique à celle de $KCOGL$ (voir la Section 5.4), mais le critère de sélection spatial est remplacé par un tirage aléatoire uniforme.

5.5.2 Algorithme d'extraction aléatoire priorisée

Pour que la comparaison avec l'approche $KCOGL$ soit rigoureuse, $KCRDKL$ doit adopter la même structure de priorisation¹¹. L'algorithme cherche en priorité à saturer le quota N avec des pires scénarios absolus. Ce n'est que si le nombre de pires scénarios est inférieur à N que l'algorithme vient piocher dans l'ensemble relaxé pour compléter sa sélection.

L'Algorithme 12 (situé en Annexe K) a une complexité en $\mathcal{O}(L \cdot T \cdot \Gamma)$, car il est notamment dominé par l'exploration du DFS susmentionné. La fonction stochastique *Shuffle* prend un temps négligeable en $\mathcal{O}(N)$.

5.5.3 Limites de l'approche $KCRDKL$

L'implémentation de $KCRDKL$ suit une logique lexicographique : 1. maximiser le coût des attaques, 2. diversifier l'information de manière stochastique. Cette heuristique constitue notre *baseline* stochastique. En la comparant à $KCOGL$, nous pourrions isoler l'impact de l'orthogonalité. Si $KCOGL$ permet au problème Maître de converger plus rapidement que $KCRDKL$ pour un même quota N , cela prouvera que la topologie spatiale des scénarios porte une information importante pour le problème. Cela justifiera ainsi l'effort computationnel lié au calcul des distances.

11. Initialement, une heuristique nommée $KCRDK$ pour *Knowledge Compilation Random K* avait été introduite pour servir de *baseline* à $KCOG$. Cependant, ayant amélioré $KCOG$ en $KCOGL$, il a été nécessaire de reproduire la transition sur $KCRDK$, mais dans un souci de simplicité, nous n'avons décrit que l'heuristique privilégiée.

6 Résultats

6.1 Protocole expérimental

Cette étude a deux objectifs : démontrer formellement la supériorité de la compilation de connaissances sur l’approche standard de Benders Adverse, puis identifier la stratégie d’extraction la plus performante selon la configuration du problème.

Le plan d’expérience s’articule autour de cinq axes. La première expérience isole le bénéfice de la factorisation matricielle en comparant *BA* et *KCU*. Le deuxième évalue la pertinence du critère de diversité en opposant l’extremum temporel (*KCUD*) à l’extremum spatial (*KCOGL*). Le troisième axe mesure le compromis entre le coût du calcul de distance spatiale (*KCOGL*) et l’efficacité d’un tirage aléatoire (*KCRDKL*). De plus, les expériences 4 et 5 permettent d’analyser les limites de l’approche stochastique *BAE_{Few}* et de mesurer l’impact de la périodicité de la demande sur la robustesse des heuristiques. Enfin, au-delà de la convergence théorique des heuristiques, une analyse stochastique par méthode de Monte Carlo permettra de certifier la robustesse de nos plans de production.

Tous les tests ont été réalisés sous Linux Ubuntu 24.04.4 avec le solveur *CPLEX* (version 22.1) sur un modèle implémenté en C++. Le jeu de données standard est constitué de 50 instances simulant une année de production ($T = 52$ périodes) pour 20 références distinctes. Une périodicité trimestrielle (13 semaines) régit la demande : la probabilité qu’une référence soit requise suit une loi de Bernoulli de paramètre $p_{demand} = 0,7$ en période dense et $(1 - p_{demand})$ en période creuse. Le budget d’incertitude Γ varie dans l’ensemble $\mathcal{D}_\Gamma = \{1, 11, \dots, 91, 101\}$. Pour la méthode probabiliste (*SelectFew*), le paramètre empirique d’acceptation d’un arc est fixé à $p_{Few} = \frac{1}{T}$.

Concernant l’espace des solutions admissibles, le polytope de production $\mathcal{X}$ est défini par deux contraintes liant les variables de production cumulée X_t . D’une part, la condition de non-décroissance ($X_t \geq X_{t-1}$). D’autre part, la contrainte de capacité maximale cumulée ($X_t \leq \mathbb{X}_t$). Pour l’ensemble de nos instances de test, la borne de capacité $\mathbb{X}_t$ a été générée aléatoirement de sorte qu’elle représente entre 80 % et 120 % de la demande cumulée nominale à la période t . Concrètement, pour chaque période, cette limite est issue d’un tirage aléatoire uniforme dans l’intervalle $[0, 8\hat{D}_t, 1, 2\hat{D}_t]$. Les performances sont évaluées selon deux métriques : le temps moyen de calcul et le nombre moyen d’itérations.

6.2 Expérience 1 : Apports de la compilation de connaissances ($N = 1$)

L’objectif de cette première expérience est d’isoler l’apport structurel de la factorisation matricielle. Pour ce faire, nous comparons *BA* avec *KCU*. Ces deux méthodes ne renvoient qu’un unique scénario optimal par itération. Nous évaluons si l’encodage d’un chemin dans un graphe orienté acyclique offre une relaxation plus forte qu’une coupe

de Benders constituée de contraintes linéaires indépendantes.

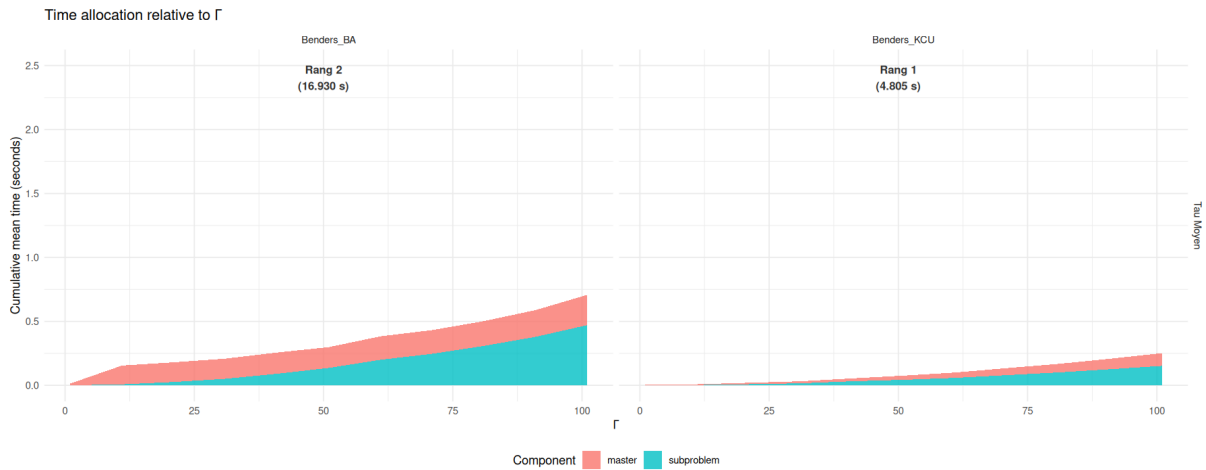


FIGURE 8 – Comparaison des temps de résolution du sous-problème et du problème maître pour *BA* & *KCU*.

L'analyse des performances, illustrée par les Figures 8 et 9, met en évidence la supériorité de l'approche *KCU*. Concernant le nombre d'itérations, *BA* croît rapidement pour plafonner autour de 7,5 itérations moyennes dès que $\Gamma \geq 20$. À l'inverse, *KCU* converge de manière presque instantanée en environ 1,5 itération, indépendamment de Γ .

La diminution du nombre d'itérations se traduit par un gain de temps relativement significatif. Le graphique de la Figure 9 montre un pic de performance autour de $\Gamma = 11$, où *KCU* résout le problème en moyenne 12 fois plus vite que *BA*. Pour des budgets d'incertitude supérieurs, le ratio se stabilise, *KCU* reste systématiquement 2 à 3 fois plus rapide.

La Figure 8 confirme la pertinence de la formulation du problème Maître de *KCU*. Pour la méthode *BA*, le temps cumulé alloué au Maître augmente avec la croissance de Γ . Cette difficulté vient du fait que le solveur doit gérer l'accumulation de contraintes linéaires au fil des itérations. La factorisation par *KC* maintient un temps de résolution marginal pour le Maître. Cette expérience valide notre conjecture : à un volume d'information extrait équivalent, l'encodage topologique de la connaissance incertaine surpasse l'approche classique.

6.3 Expérience 2 : Extremum temporel vs. extremum spatial ($N = 2$)

En fixant le quota d'extraction à $N = 2$, cette expérience évalue quelle définition de la diversité engendre les coupes les plus efficaces. Nous opposons l'approche *KCOGL* à l'heuristique *KCUD*. La méthode aléatoire *KCRDKL*($N = 2$) sert de *baseline* stochastique. L'espace d'exploration du DFS (pour *KCOGL* et *KCRDKL*) varie dans $\mathcal{D}_L = \{100, 200, \dots, 1500\}$.

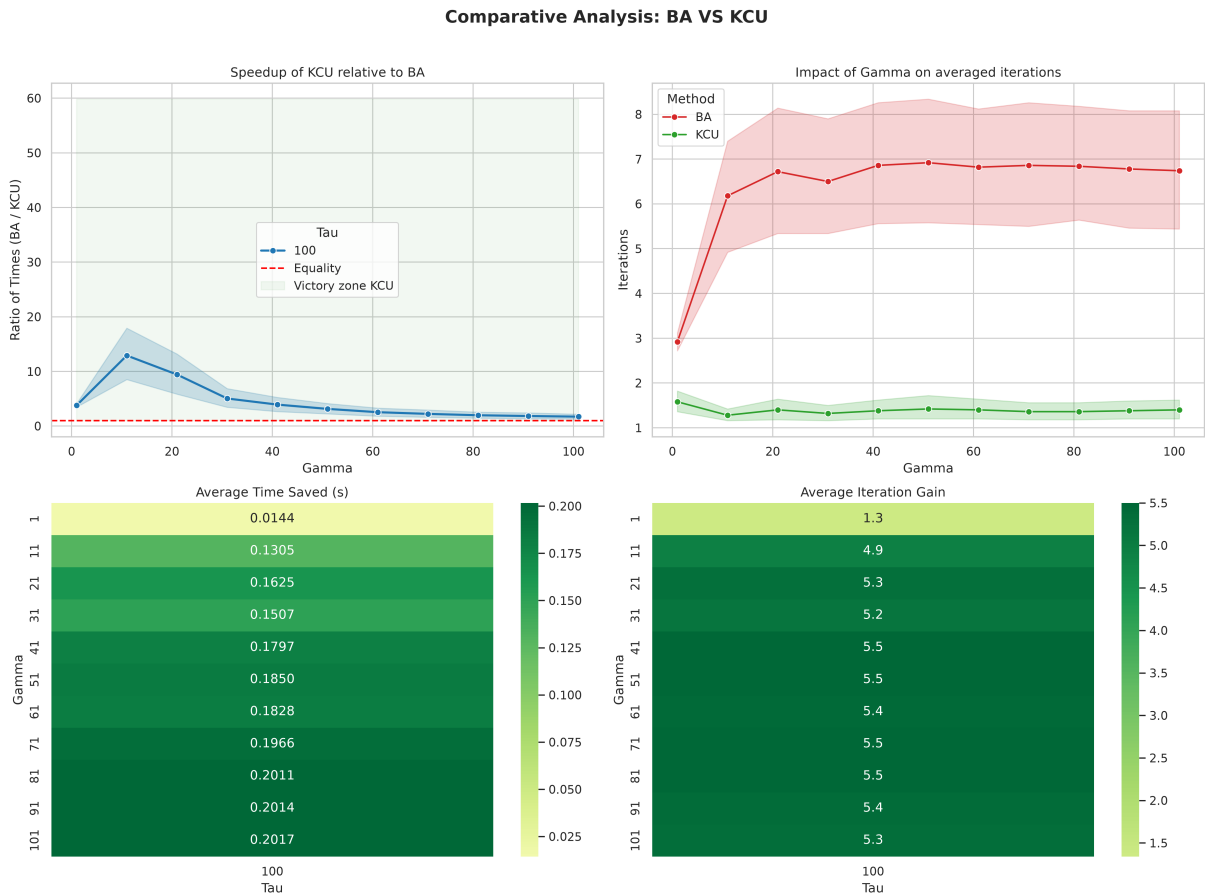


FIGURE 9 – Analyse comparative entre *BA* & *KCU*.

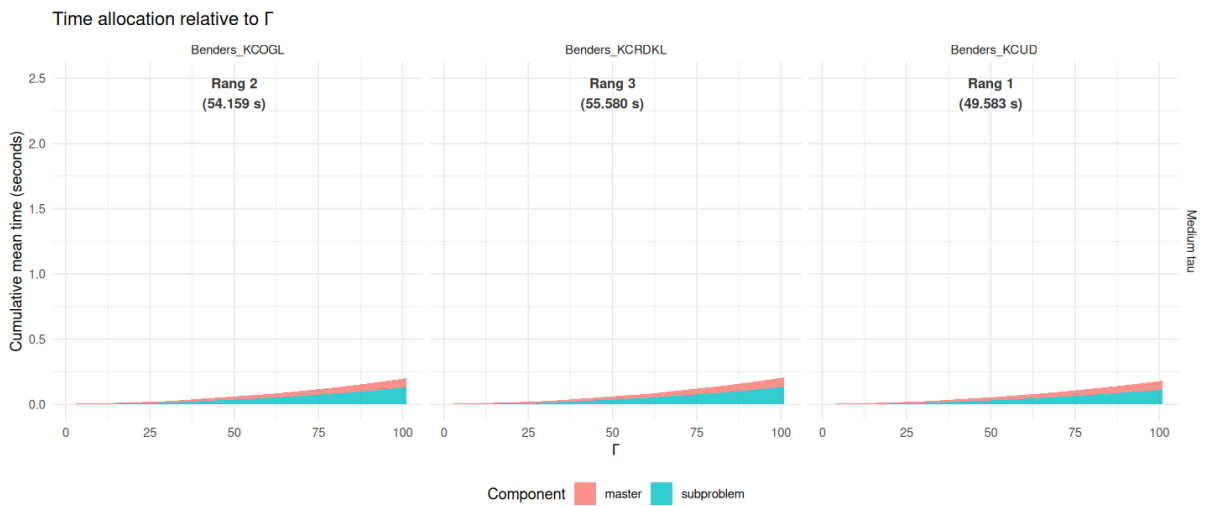
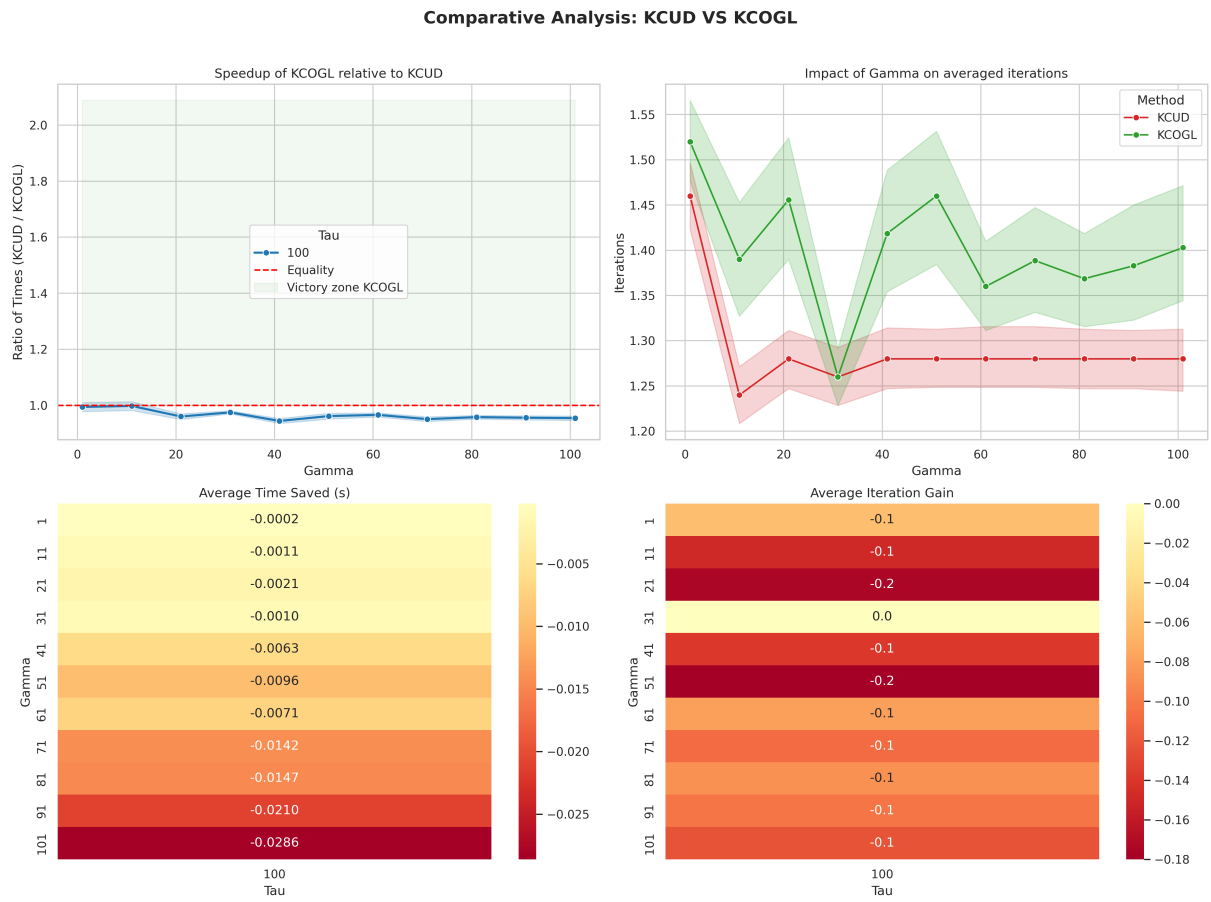


FIGURE 10 – Comparaison des temps de résolution du sous-problème et du problème maître pour *KCUD*, *KCOGL* & *KCRDKL*.

FIGURE 11 – Analyse comparative entre *KCUD* & *KCOGL*.

Les résultats confirment la supériorité de l’heuristique *KCUD* pour $N = 2$. L’analyse des temps cumulatifs moyens (Figure 10) établit le classement suivant : *KCUD* ($\approx 49,58$ s) domine *KCOGL* ($\approx 54,15$ s) et *KCRDKL* ($\approx 55,58$ s). Au-delà de son algorithme d’extraction glouton, intrinsèquement plus léger qu’un DFS, *KCUD* réduit également le temps de résolution et le nombre d’itérations du problème Maître (Figure 11).

Cette performance s’explique par la structure même du problème de *Lot-Sizing*. Contrairement à une dispersion spatiale, *KCUD* force l’adversaire à évaluer les extrêmes temporels de la consommation du budget d’incertitude. En planification de production, ces périodes sont critiques pour la gestion des stocks et les coûts de lancement. La diversité temporelle génère ainsi des coupes plus discriminantes en ciblant directement les goulots d’étranglement. Cela rend l’approche *KCUD* optimale pour $N = 2$.

6.4 Expérience 3 : Apports de la distance ($N > 2$)

L’heuristique d’orthogonalité repose sur un compromis théorique : le surcoût computationnel lié au calcul des distances dans le sous-problème doit être compensé par une convergence plus rapide du Maître. À l’inverse, l’approche stochastique ne perd pas de temps avec cela, garantissant un temps de sous-problème stable, au risque d’augmenter le nombre d’itérations globales.

Dans cette expérience, le nombre de chemins extraits varie dans $\mathcal{D}_N = \{3, 5, 7, 9, 11\}$. L’analyse de la distribution du ratio des temps moyens (Figure 12) démontre que la méthode aléatoire *KCRDKL* est globalement plus rapide que *KCOGL*. La Figure 13 explique ce phénomène : le temps alloué au sous-problème de *KCOGL* croît de façon importante avec N .

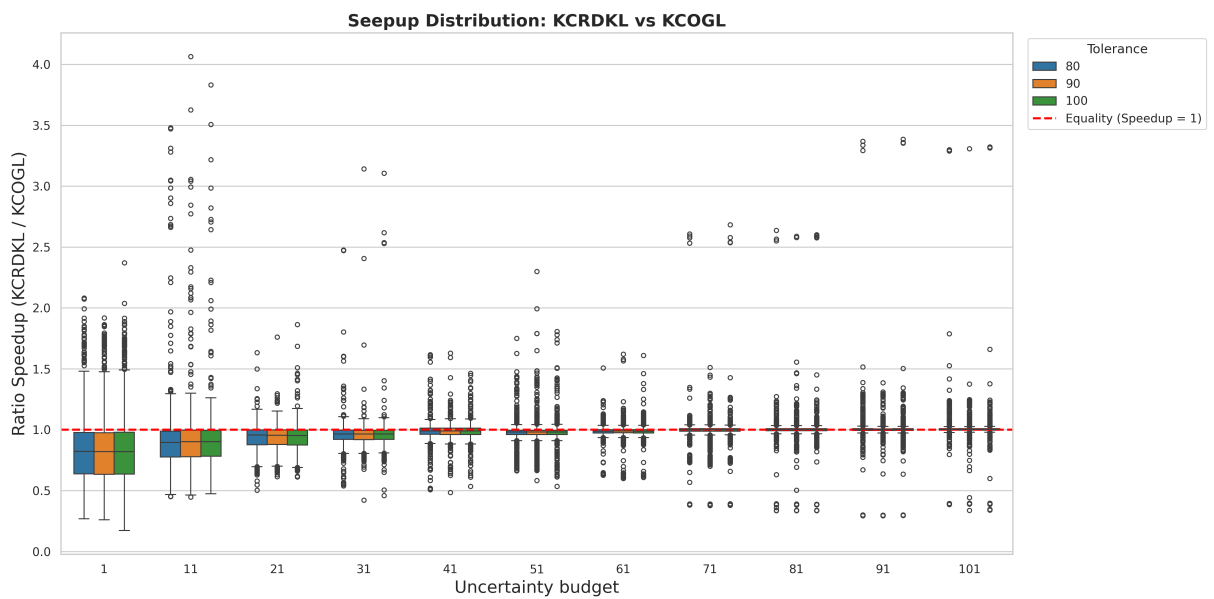


FIGURE 12 – Accélération de la distribution entre *KCRDKL* & *KCOGL*.

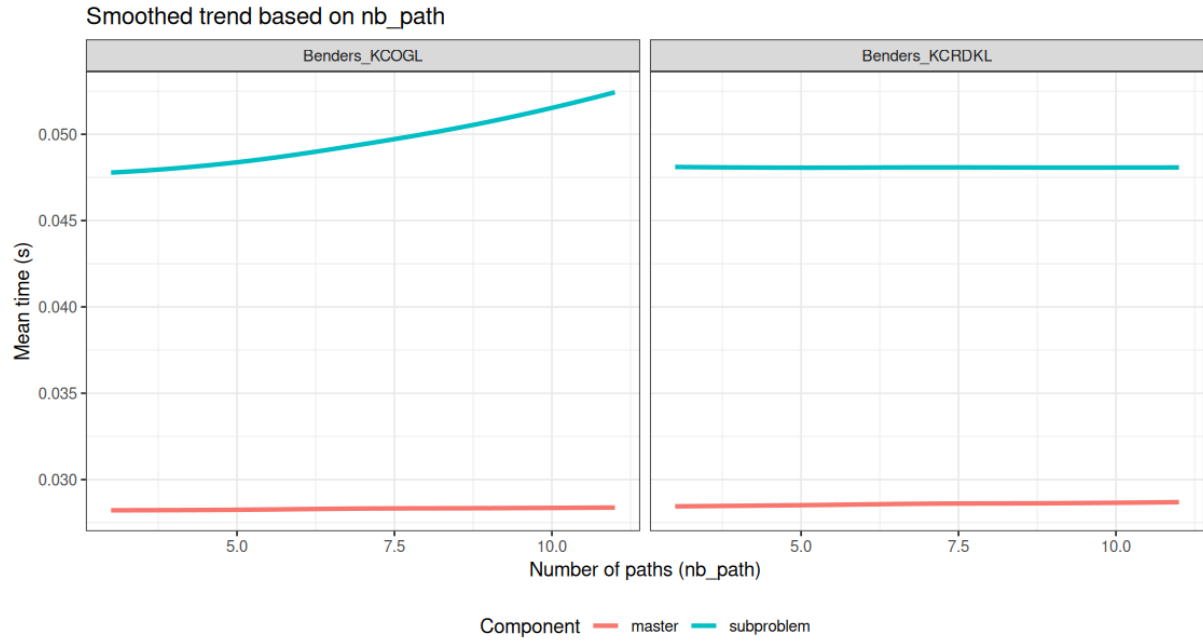


FIGURE 13 – Progression lissée entre *KCRDGL* & *KCOGL* par rapport à N .

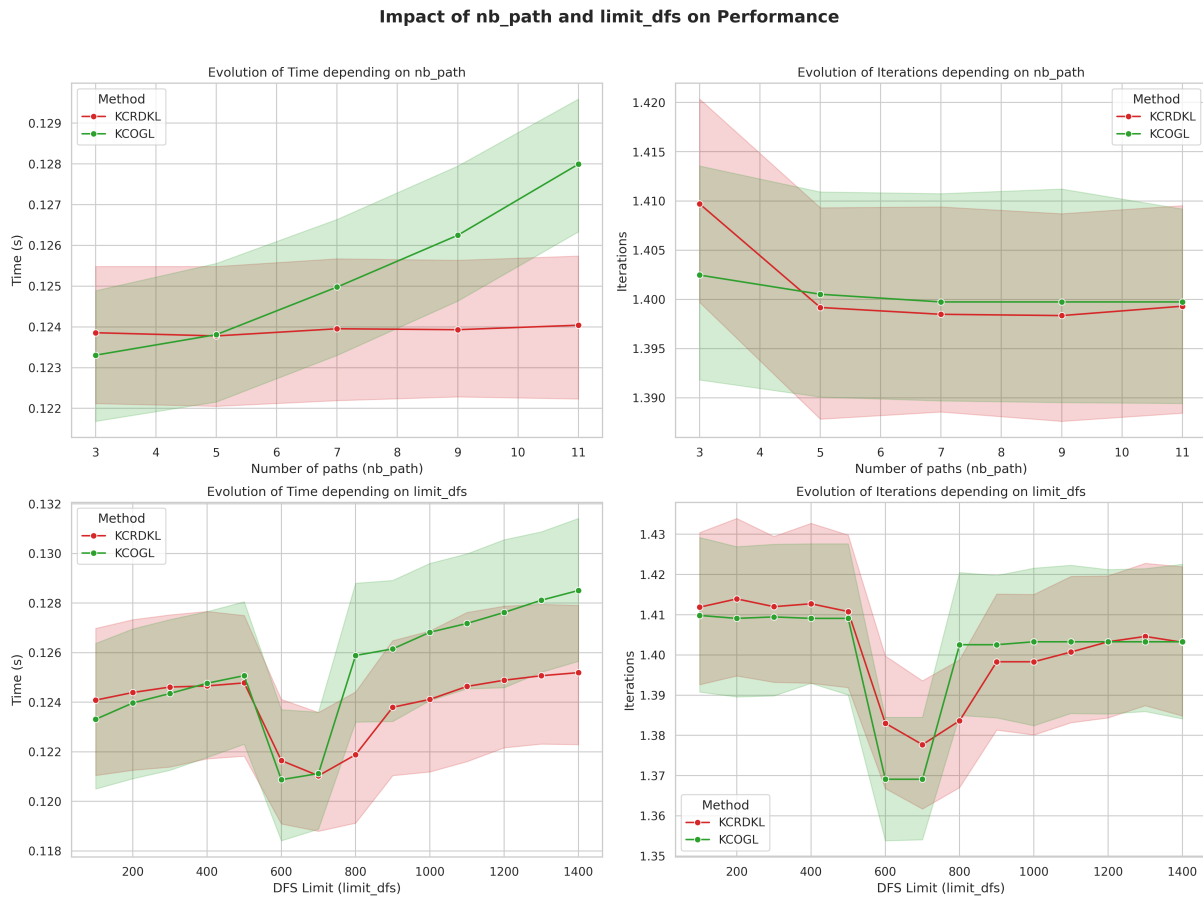


FIGURE 14 – Impact de N et L sur les performances.

Bien que la Figure 14 démontre que l’orthogonalité spatiale permet parfois de réduire marginalement les itérations du Maître, ce gain est relativement faible et ne compense pas le coût de l’extraction. En conclusion, pour $N > 2$, le tirage aléatoire priorisé constitue le meilleur compromis de résolution ; la topologie du problème de *Lot-Sizing* ne justifie pas la complexité d’un calcul exhaustif des distances de Manhattan.

6.5 Expériences 4 et 5 : Limites de BAE et impact de la périodicité

La quatrième expérience évalue l’heuristique probabiliste BAE_{Few} (notée $BAEF$), proposée par [13]. Si cette méthode surpasse l’approche standard BA sur de petits budgets, elle devient moins performante face à des incertitudes importantes (Figure 15). Son sous-problème est très rapide, mais l’absence de limite sur le nombre de scénarios extraits sature le Maître. Les approches que nous avons développées (KCU , $KCUD$, $KCRDKL$) corrigent ce défaut en imposant un quota N . Cela offre des performances supérieures. KCU et $KCUD$ restent les heuristiques à privilégier.

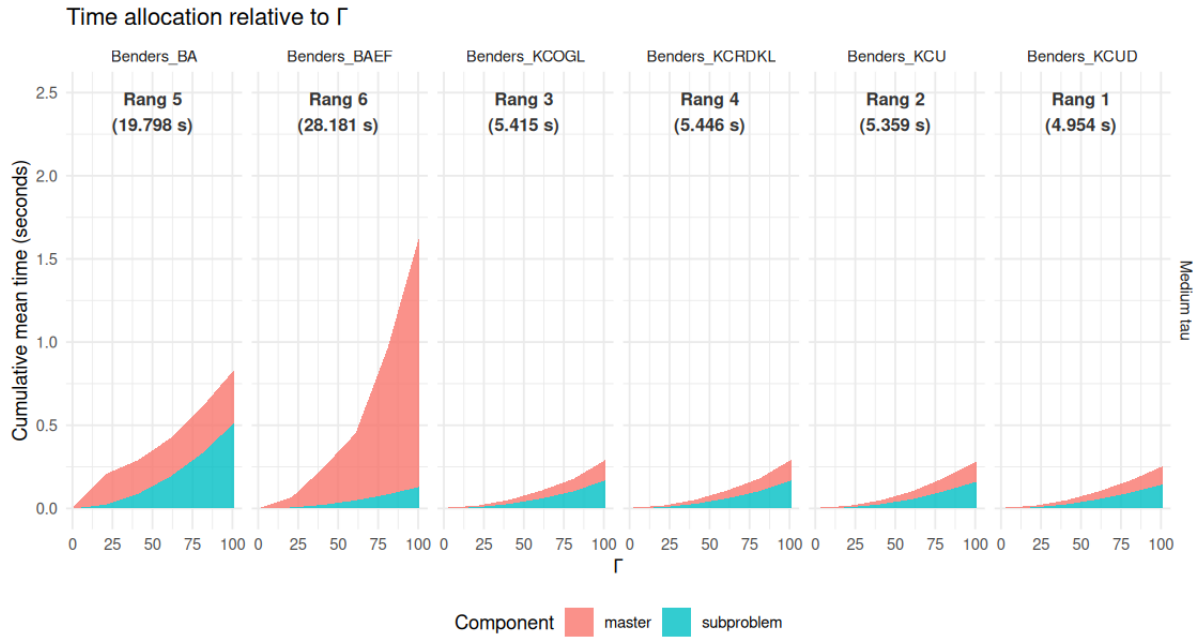


FIGURE 15 – Comparaison des temps de résolution du sous-problème et du problème maître avec périodicité.

La dernière expérience mesure l’impact de la périodicité de la demande. Les résultats (Figure 16) montrent que la suppression de la saisonnalité simplifie la résolution globale. Les performances des heuristiques BA et BAE_{Few} sont augmentées. Néanmoins, le classement des performances reste inchangé, prouvant la robustesse de nos algorithmes factorisés face aux variations de la structure de la demande.

Γ	<i>BA</i>		<i>BAE_{Few}</i>		<i>KCU</i>		<i>KCUD</i>		<i>KCRDKL</i>		<i>KCOGL</i>	
	time	iter.	time	iter.	time	iter.	time	iter.	time	iter.	time	iter.
1	0,015	2,96	0,004	1,28	0,004	1,68	0,004	1,44	0,005	1,56	0,005	1,50
21	0,208	8,28	0,073	1,0	0,023	1,32	0,023	1,26	0,025	1,34	0,024	1,34
41	0,292	8,40	0,271	1,0	0,079	1,32	0,079	1,26	0,084	1,34	0,083	1,34
61	0,422	8,24	0,507	1,0	0,173	1,32	0,167	1,26	0,176	1,32	0,176	1,32
81	0,610	8,28	1,057	1,0	0,314	1,48	0,281	1,26	0,297	1,32	0,296	1,32
101	0,833	8,48	1,798	1,0	0,498	1,50	0,436	1,26	0,506	1,50	0,502	1,50

TABLE 2 – Extrait du temps moyen et du nombre moyen d'itérations des 6 méthodes pour $\Gamma \in \{1, 21, 41, 61, 81, 101\}$. Les valeurs sont arrondies au millièrme.

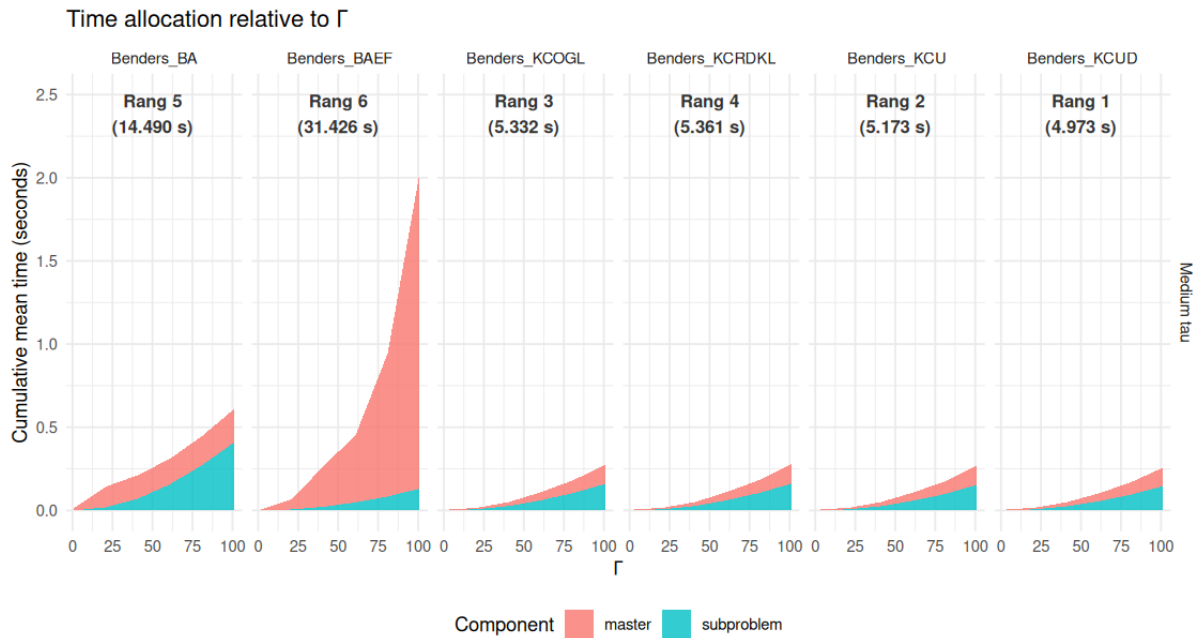


FIGURE 16 – Comparaison des temps de résolution des sous-problèmes et du problème Maître sans périodicité.

6.6 Validation de nos modèles par la méthode de Monte-Carlo

La résolution du problème de *Lot-Sizing* par la décomposition de Benders nous garantit l'obtention d'un plan de production mathématiquement robuste contre le pire scénario d'incertitude. Néanmoins, dans un contexte industriel réel, la probabilité que la demande suive exactement cette configuration est faible. Afin d'évaluer la robustesse empirique et la viabilité économique de nos plans de production, nous avons complété notre étude par une simulation stochastique via une approche de Monte-Carlo.

6.6.1 Protocole expérimental

Le principe consiste à figer le plan de production optimal X^* généré par notre heuristique, puis à l'évaluer face à un très grand nombre de scénarios de demande aléatoires. Pour garantir la validité de l'expérience, chaque scénario est construit en suivant strictement toutes les contraintes du problème.

L'instance avec laquelle nous avons obtenu X^* a été générée de la façon suivante. Elle utilise le même protocole expérimental que celui de la Section 6.1. Cependant, nous avons fixé $\Gamma = 101$ et $\tau = 0,9$. Pour chaque heuristique testée (*KCU*, *KCUD*, *KCRDKL* et *KCOGL*), nous avons simulé $N_{MC} = 10\,000$ scénarios aléatoires indépendants et identiquement distribués. Nous considérons que N_{MC} est assez grand pour pouvoir utiliser les théorèmes suivants.

Sous l'hypothèse de moyenne finie, la loi forte des grands nombres (LGN) nous assure que la moyenne empirique des coûts calculés sur ces N_{MC} scénarios converge presque sûrement vers le paramètre théorique cible $\mu = \mathbb{E}[\text{Coût}(X^*, \xi)]$, qui représente l'espérance mathématique du coût réel de l'usine. En pratique, ce paramètre inconnu μ , que nous noterons *EAC* pour *Expected Average Cost*, est approché par son estimateur consistant $\widehat{EAC}$ défini par l'Équation (6.1).

$$\widehat{EAC} = \frac{1}{N_{MC}} \sum_{k=1}^{N_{MC}} \text{Coût}(X^*, \xi_k) \quad (6.1)$$

Sous l'hypothèse de variance finie, nous pouvons utiliser le théorème central limite (TCL). Indépendamment de la loi de distribution initiale de l'incertitude, le TCL nous permet d'approximer la distribution de cet estimateur par une loi Normale. Nous pouvons donc construire un intervalle de confiance à 95 % (voir l'Équation (6.2)) autour de l'estimateur $\widehat{EAC}$ en utilisant l'écart-type empirique¹², noté $\hat{\sigma}$. Cela permettra de vérifier statistiquement que notre solution est stable.

$$CI_{95\%} = \left[\widehat{EAC} \pm 1,96 \frac{\hat{\sigma}}{\sqrt{N_{MC}}} \right] \quad (6.2)$$

12. Ceci est autorisé par le théorème de Slutsky. La statistique $\sqrt{N_{MC}}(\widehat{EAC} - \mu)/\sigma$ converge en loi vers $\mathcal{N}(0,1)$ (TCL), et $\hat{\sigma}$ converge en probabilité vers σ (LGN). Le théorème garantit alors que remplacer σ par $\hat{\sigma}$ ne change pas la loi limite.

6.6.2 Analyse de la robustesse

Le Tableau 3 synthétise les résultats statistiques obtenus lors de cette simulation.¹³

	<i>KCU</i>	<i>KCUD</i>	<i>KCRDKL</i>	<i>KCOGL</i>
$\widehat{EAC}$	9 828,08	9 828,94	9 828,37	9 827,92
$\hat{\sigma}$	40,24	40,15	40,08	39,80
CI _{95 %}	[9 827, 30; 9 828, 87]	[9 828, 15; 9 829, 73]	[9 827, 58; 9 829, 15]	[9 827, 14; 9 828, 71]
WCS	9 974,20	9 965,08	9 977,12	9 975,72
TWC	10 034	10 034	10 034	10 034

TABLE 3 – Analyse de la robustesse d’un plan de production par la méthode de Monte-Carlo (*KCU*, *KCUD*, *KCRDKL* et *KCOGL*). On a : *EAC* pour « *expected average cost* » ; *WCS* pour « *worst-case simulated* » et *TWC* pour « *theoretical worst-case* ».

L’analyse de ces données valide empiriquement la pertinence de notre modèle d’optimisation robuste.

Premièrement, nous constatons que, pour toutes les heuristiques, le pire cas simulé aléatoirement (WCS) est strictement inférieur au pire cas théorique (TWC) de 10 034, garanti par l’algorithme de Benders. Le simulateur, sur 10 000 itérations, n’a jamais pu trouver une combinaison d’incertitudes capable de dépasser la borne supérieure établie par le problème Maître. Cela certifie que l’approche adverse identifie bien la borne supérieure absolue du problème. Deuxièmement, l’écart-type ($\sigma \approx 40$) est relativement faible au regard des coûts totaux. Les intervalles de confiance à 95 % sont très serrés. Cette faible variance prouve que les plans de production générés par nos heuristiques sont industriellement stables. Enfin, le coût moyen espéré ($EAC \approx 9 828$) illustre le concept du « prix de la robustesse ». Cet indicateur mesure la perte de performance que l’on accepte de payer pour se protéger contre l’incertitude. Pour éviter une perte maximale théorique de 10 034, le modèle prévoit des stocks de sécurité. Cela génère un coût quotidien moyen qui est légèrement inférieur. Même si les méthodes *KCOGL* et *KCU* semblent donner des plans un peu moins coûteux, les résultats montrent que nos méthodes aboutissent à des solutions robustes face à l’incertitude.

6.7 Conclusion & Pistes d’améliorations

Cette étude a présenté et évalué plusieurs heuristiques de compilation de connaissances pour la résolution robuste de problèmes de *Lot-Sizing* via la décomposition de Benders. L’objectif de cette analyse était d’identifier la stratégie d’extraction des scénarios adverses la plus efficace, en termes de compromis entre la qualité des coupes générées et le coût computationnel associé.

Après avoir réalisé plusieurs expériences, nous pouvons tirer trois conclusions quant à la manière la plus efficace de modéliser et d’extraire l’information adverse. L’intégra-

13. Pour plus de lisibilité, les résultats des heuristiques de référence (*BA*, *BAE_{All}* et *BAE_{Few}*) sont consultables dans le Tableau 4 (situé en Annexe L).

tion des pires scénarios sous forme de chemins au sein d'un graphe partiel de budget permet au problème Maître de limiter l'explosion combinatoire. L'expérience menée sur $N = 2$ scénarios a démontré que forcer l'adversaire aux extrêmes temporels génère des coupes plus discriminantes. Même si l'approche *KCOGL* nous semblait intéressante d'un point de vue mathématique, elle s'est révélée sous-optimale, apportant un gain total très marginal. A l'inverse, la baseline stochastique *KCRDKL* s'impose comme une méthode plus robuste pour des quotas de sélection plus larges ($N > 2$).

La limite des méthodes aléatoires BAE_{Few} et *KCRDKL* est qu'elles sont « aveugles ». En tirant les scénarios de manière uniforme dans l'ensemble des chemins candidats, nous traitons toutes les attaques de la même façon. Or, dans la réalité, certaines configurations d'incertitude ont beaucoup plus de chances de dégrader la valeur objective du Maître que d'autres.

De plus, comme nous l'avons dit en introduction, l'apprentissage automatique est de plus en plus présent; nous pouvons y voir une ouverture vers l'Apprentissage par Renforcement (*Reinforcement Learning* - *RL*). L'idée serait de remplacer ce tirage uniforme par un agent intelligent capable d'apprendre une politique de sélection $\pi(a|s)$. Concrètement, le modèle s'articulerait de la manière suivante. L'état s serait représenté par la topologie du graphe de budget partiel généré par une des heuristiques. L'action a consisterait à cibler et à pondérer les scénarios adverses les plus pertinents parmi les candidats. Enfin, le score final serait mesuré par la capacité de la coupe de Benders générée à dégrader fortement la borne inférieure du Maître.

Étant donné que notre graphe de budget est un DAG, l'utilisation d'un Réseau de Neurones Graphiques (GNN) comme approximateur de fonction de valeur serait pertinente. Les GNN sont conçus pour traiter ces structures de données non-euclidiennes et pourraient encoder les contraintes de capacité locales. Contrairement à *KCOGL*, nous nous affranchissons de la complexité en $\mathcal{O}(N^2)$ liée au calcul des distances L_1 . Contrairement à *KCRDKL*, l'extraction n'est plus aléatoire. L'agent apprendrait à identifier les scénarios critiques, couplant ainsi la vitesse d'extraction de la compilation de connaissances avec l'intelligence de l'apprentissage automatique.

Une sixième expérience consisterait à modéliser une variante du problème standard de Benders Adverse. Elle renverrait N scénarios au Maître par itération, mais sans utiliser la compilation de connaissances. Si, au lieu de fusionner les différents graphes de budget, nous concaténons simplement les différents scénarios dans une liste, nous pourrions alors vérifier si la factorisation matricielle est plus importante que l'action de ramener plusieurs scénarios par itération. Lors du *backtracking*, pour générer ces N scénarios, nous pourrions utiliser un algorithme des K -plus longs chemins en adaptant le *backtracking* par un algorithme de type *Yen*. Formellement introduit par Jin Y. Yen [16], il s'agit d'un algorithme de recherche du plus court chemin. Cependant, en multipliant le coût des arcs par (-1) , il permet de trouver les plus longs chemins dans le DAG (cette approche a notamment été développée par Eppstein dans [7]). Nous pourrions également intégrer directement des heuristiques de calcul de distance. Nous conjecturons que cette approche ne sera pas optimale. En effet, la taille du Maître va croître, car

ajouter N scénarios par itération implique d'ajouter $\mathcal{O}(NT)$ variables et contraintes au modèle *CPLEX*. Enfin, bien que nous économisions du temps en limitant le nombre d'itérations, le temps de résolution risque d'exploser. Nous testerons cette hypothèse avant la fin du stage.

7 Conclusion

Dans cette dernière section, nous ferons le point sur le travail qui a été réalisé durant ce stage. Puis, nous détaillerons les apports techniques et professionnels associés.

7.1 Synthèse sur le travail effectué

Ce stage a eu pour objectif principal l'étude et l'amélioration des méthodes d'optimisation robuste appliquées au problème de *Lot-Sizing* sous incertitude quant à la demande cumulée.

Le travail s'est d'abord concentré sur la restructuration d'un modèle informatique pour le rendre stable et fonctionnel. Face au constat que l'approche probabiliste souffrait d'une explosion combinatoire, le stage a eu pour but de concevoir, d'implémenter et d'évaluer de nouvelles heuristiques d'extraction de scénarios. Les différentes expériences ont permis de démontrer la supériorité et la robustesse de la factorisation des connaissances, notamment par le biais de l'heuristique *KCUD*.

7.2 Apports professionnels

Sur le plan professionnel, cette expérience m'a permis de découvrir de l'intérieur le fonctionnement d'un laboratoire de recherche, ses niveaux d'exigence et cette effervescence intellectuelle.

La confiance que m'ont accordée Romain Guillaume et Christian Artigues m'a permis d'acquérir une certaine autonomie quant à la gestion de l'avancement d'une recherche. J'ai pu m'approprier un sujet complexe en justifiant mes choix mathématiques face à eux. Dans le même sens, ce stage a également renforcé ma rigueur et mon formalisme mathématique, indispensables pour modéliser correctement les problèmes. Au-delà des compétences techniques, l'effervescence de la recherche a confirmé mon souhait de m'investir plus tard dans ce domaine.

De plus, ces résultats sont prometteurs et ouvrent la voie à une publication dans une conférence en recherche opérationnelle. Être coauteur sera bénéfique pour commencer une carrière en recherche.

7.3 Apports techniques

Techniquement, ce stage s'est avéré formateur. D'un point de vue informatique, j'ai considérablement renforcé mes compétences en développement C++ ainsi qu'en modélisation *CPLEX*.

Sur le plan théorique, j'ai approfondi mes connaissances en optimisation combinatoire, en théorie des graphes ainsi qu'en programmation dynamique. J'ai également pu m'approprier la méthode de Benders, heuristique qui nous sera enseignée en deuxième année de master.

Enfin, dans la continuité de cette première année de master en recherche opérationnelle, j'ai approfondi mes compétences en recherche. Notamment sur la capacité d'effectuer une veille bibliographique et sur l'élaboration de protocoles expérimentaux.

7.4 Perspectives

Ce stage ouvre plusieurs perspectives. Comme évoqué précédemment, la transition vers des modèles d'apprentissage automatique constitue la suite logique de ces travaux. Après avoir échangé longuement avec des doctorant·e·s du laboratoire, la possibilité d'effectuer une thèse dans la continuité de cette thématique ou sur un nouveau sujet est une perspective intéressante que je laisserai mûrir durant l'année prochaine.

Utilisation de l'intelligence artificielle. Différents outils d'IA ont été utilisés durant ce stage à des fins d'aide à la compréhension de notions complexes et d'aide à la correction de code, mais n'ont pas été moteurs de la réflexion.

Références

- [1] Richard BELLMAN. *Dynamic Programming*. Princeton, NJ : Princeton University Press, 1957. ISBN : 9780691146683.
- [2] J.F. BENDERS. « Partitioning procedures for solving mixed-variables programming problems. » In : *Numerische Mathematik* 4 (1962), p. 238-252. DOI : [10.1007/BF01386316](https://doi.org/10.1007/BF01386316). URL : <http://eudml.org/doc/131533>.
- [3] David BERGMAN et al. *Decision Diagrams for Optimization*. Springer, 2016.
- [4] Dimitris BERTSIMAS et Melvyn SIM. « The Price of Robustness ». In : *Operations Research* 52 (fév. 2004), p. 35-53. DOI : [10.1287/opre.1030.0065](https://doi.org/10.1287/opre.1030.0065).
- [5] Gabriel R. BITRAN et Horacio H. YANASSE. « Computational Complexity of the Capacitated Lot Size Problem ». In : *Management Science* 28.10 (oct. 1982), p. 1174-1186. DOI : [10.1287/mnsc.28.10.1174](https://doi.org/10.1287/mnsc.28.10.1174). URL : <https://ideas.repec.org/a/inm/ormnsc/v28y1982i10p1174-1186.html>.
- [6] A. DARWICHE et P. MARQUIS. « A Knowledge Compilation Map ». In : *Journal of Artificial Intelligence Research* 17 (2002), p. 229-264. ISSN : 1076-9757. DOI : [10.1613/jair.989](https://doi.org/10.1613/jair.989). URL : <http://dx.doi.org/10.1613/jair.989>.
- [7] David EPPSTEIN. « Finding the k Shortest Paths ». In : *SIAM Journal on Computing* 28 (juill. 2006), p. 652-673. DOI : [10.1137/S0097539795290477](https://doi.org/10.1137/S0097539795290477).
- [8] R. GUILLAUME, A. KASPERSKI et P. ZIELINSKI. *Robust production planning with budgeted cumulative demand uncertainty*. 2020. DOI : [10.48550/arXiv.2009.05736](https://doi.org/10.48550/arXiv.2009.05736). arXiv : [2009.05736](https://arxiv.org/abs/2009.05736) [cs.DS]. URL : <https://arxiv.org/abs/2009.05736>.
- [9] R. W. HAMMING. « Error detecting and error correcting codes ». In : *The Bell System Technical Journal* 29.2 (avr. 1950), p. 147-160. ISSN : 0005-8580. DOI : [10.1002/j.1538-7305.1950.tb00463.x](https://doi.org/10.1002/j.1538-7305.1950.tb00463.x).
- [10] Hermann MINKOWSKI. *Geometrie der Zahlen*. Leipzig und Berlin : B.G. Teubner, 1910, p. 256.
- [11] A. MUTAPCIC et S. BOYD. « Cutting-set methods for robust convex optimization with pessimizing oracles ». In : *Optimization Methods and Software* 24.3 (2009), p. 381-406. DOI : [10.1080/10556780802712889](https://doi.org/10.1080/10556780802712889).
- [12] J. PÄTZOLD et A. SCHÖBEL. « Approximate cutting plane approaches for exact solutions to robust optimization problems ». In : *European Journal of Operational Research* 284.1 (2020), p. 20-30. ISSN : 0377-2217. DOI : [10.1016/j.ejor.2019.11.059](https://doi.org/10.1016/j.ejor.2019.11.059). URL : <https://www.sciencedirect.com/science/article/pii/S0377221719309713>.
- [13] T. PORTOLEAU. « Représentations discrètes pour l'ordonnancement et la planification robustes ». Theses. Université toulouse Jean Jaurès, juin 2022. URL : <https://laas.hal.science/tel-04631599>.

- [14] T. PORTOLEAU, R. GUILLAUME et C. ARTIGUES. « Une variante de la méthode de Benders adverse pour le problème de lot-sizing robuste avec budget d'incertitude ». In : *23ème congrès annuel de la Société Française de Recherche Opérationnelle et d'Aide à la Décision*. INSA Lyon. Villeurbanne - Lyon, France, fév. 2022. URL : <https://hal.science/hal-03595446>.
- [15] A. L. SOYSTER. « Convex Programming with Set-Inclusive Constraints and Applications to Inexact Linear Programming ». In : *Operations Research* 21.5 (1973), p. 1154-1157. DOI : [10.1287/opre.21.5.1154](https://doi.org/10.1287/opre.21.5.1154).
- [16] Jin Y. YEN. « Finding the K Shortest Loopless Paths in a Network ». In : *Management Science* 17.11 (juill. 1971), p. 712-716. DOI : [10.1287/mnsc.17.11.712](https://doi.org/10.1287/mnsc.17.11.712). URL : <https://ideas.repec.org/a/inm/ormnsc/v17y1971i11p712-716.html>.

Annexes

Sommaire des annexes

A	Définition d'un graphe de budget	41
B	Définition d'un graphe partiel de budget	42
C	Algorithme de programmation dynamique du problème Adv (BAE)	43
D	Algorithme d'initialisation du budget d'incertitude	44
E	Algorithme d'extraction du scénario optimal (KCU)	45
F	Algorithmes d'extraction des scénarios optimaux (KCUD)	46
G	Algorithme récursif d'extraction des scénarios (DFS)	48
H	Schéma du fonctionnement de l'algorithme DFS	49
I	Algorithme de discrétisation MaxMin (KCOG, KCOGL)	50
J	Algorithme complet d'extraction (KCOGL)	51
K	Algorithme d'extraction priorisé (KCRDKL)	52
L	Analyse de la robustesse d'un plan de production par Monte Carlo	53
M	Déroulement de l'algorithme de Benders Adverse	54
	M.1 Déroulement de l'algorithme	54
	M.1.1 Formulation du problème	54
	M.1.2 Itérations	54
	M.2 Résultats de l'expérimentation par CPLEX	57
N	Reproducing Robust Instance Scheduling	58
	N.1 Launch of the Experiment	58
	N.2 Test Different Heuristics	59
	N.3 Instance Generation	60
	N.4 Statistical Analysis of the Results	61

A Définition d'un graphe de budget

Définition A.1: Graphe de budget

On note ce graphe $\mathcal{G} = (V, A)$. L'ensemble des nœuds V partitionné en $T + 2$ niveaux et, mis à part le premier et le dernier niveau qui contiennent un unique nœud chacun, chaque niveau contient $\Gamma + 1$ nœuds. On note respectivement v_0 et v_{T+1} les nœuds du premier et du dernier niveau, et $v_{t,i}$ pour $1 \leq t \leq T$ et $0 \leq i \leq \Gamma$ tous les autres nœuds du graphe. L'ensemble des arcs A est construit de sorte que l'arc de $v_{t-1,i}$ vers $v_{t,j}$ existe et si et seulement si $i \leq j$ et $j - i \leq \Delta_t$ pour chaque t (ou niveau) compris entre 1 et T , et un arc de $v_{t,i}$ vers v_{T+1} est défini pour i compris entre 0 et Γ . Ce graphe est tel que chaque chemin de v_0 vers v_{T+1} représente un scénario de $\mathcal{U}$, et que si un nœud $v_{t,i}$ est sur ce chemin, alors i unités de budget ont été consommées entre la première période et la période t . Ensuite, on définit les coûts des arcs de sorte qu'ils représentent chacun la contribution à la fonction objectif de la décision de faire varier la demande à une période donnée pour une solution X donnée. Plus formellement, on définit les coûts des arcs par :

$$c_{t-1,i,t,j} = \begin{cases} \max\{c^I(X_t - (\hat{\mathcal{D}}_t - (j - i))), \\ c^B(\hat{\mathcal{D}}_t + (j - i) - X_t)\} + c^P y_t & \text{si } 1 \leq t \leq T - 1, \\ \max\{c^I(X_t - (\hat{\mathcal{D}}_t - (j - i))) - b^P(\hat{\mathcal{D}}_t - (j - i)), \\ c^B(\hat{\mathcal{D}}_t + (j - i) - X_t) - b^P X_t\} + c^P y_t & \text{si } t = T, \\ 0 & \text{si } t = T + 1 \end{cases}$$

où $c_{t-1,i,t,j}$ est le coût de l'arc entre les nœuds $v_{t-1,i}$ et $v_{t,j}$ lorsque cet arc existe, et $y_t = 0$ si $X_{t-1} = X_t$ et 1 sinon.

Source [8].

B Définition d'un graphe partiel de budget

Définition B.1: Graphe partiel de budget

Soit $\mathcal{G} = (V, A)$ un graphe de budget défini suivant la Définition A.1. On dit que $\mathcal{G}' = (V, A')$ est un graphe partiel de budget de $\mathcal{G}$ si $A' \subset A$ et, pour tout arc $a \in A'$, il existe un chemin de v_0 vers v_{T+1} passant par l'arc a . Les poids des arcs de ce graphe sont calculés de la même manière que pour le graphe défini précédemment. On a alors :

$$\mathcal{U}_{\mathcal{G}'} \subset \mathcal{U}_{\mathcal{G}}.$$

Source [13].

C Algorithme de programmation dynamique du problème **Adv** (*BAE*)

Algorithm 4: Adv_subGraph

Require: $\mathcal{G} = (V, A)$ and $\tau \in [0, 1]$
Ensure: $\mathcal{G}' = (V, A'), \pi_{T+1}$

```

1  $\pi_0 \leftarrow 0$ 
2 for  $t = 1 \rightarrow T + 1$ 
3   for  $j = 0 \rightarrow \Gamma + 1$ 
4      $\pi_{t,j} \leftarrow \max\{\pi_{t-1,i} + c_{t-1,i,t,j} \mid (v_{t-1,i}, v_{t,j}) \in A\}$ 
5  $A' \leftarrow \{\}$ 
6 for  $t = T \rightarrow 1$ 
7   for  $j = 0 \rightarrow \Gamma + 1$ 
8     if  $\pi_{T,j} \geq \tau \pi_{T+1}$  and  $t = T$ 
9        $A' \leftarrow A' \cup \{(v_{T,j}, v_{T+1})\}$ 
10       $v_{T,j} \leftarrow \text{marqued}$ 
11    if  $v_{t,j}$  marqued
12      for  $i = 0 \rightarrow \Gamma + 1$ 
13        if  $\pi_{t-1,i} + c_{t-1,i,t,j} = \pi_{t,j}$ 
14           $A' \leftarrow A' \cup \{(v_{t-1,i}, v_{t,j})\}$ 
15           $v_{t-1,i} \leftarrow \text{marqued}$ 
16 return  $\mathcal{G}' = (V, A'), \pi_{T+1}$ 

```

D Algorithme d'initialisation du budget d'incertitude

Algorithm 5: Proportional Initialization of Uncertainty Bounds

Require: Total budget Γ , Adverse margin α , Marginal requests d , Total demand

D_{total}

Ensure: Proportional uncertainty bounds Δ

```

1 for  $t = 1$  to  $T$ 
2   if  $d_t == 0$ 
3      $\Delta_t \leftarrow 0$ 
4   else
5      $p_t \leftarrow \frac{d_t}{D_{total}}$       // Share of demand in period  $t$  relative to total
      demand
6      $\Delta_t \leftarrow \lceil (\Gamma \times \alpha) \times p_t \rceil$       // Proportional budget allocation
7     if  $\Delta_t > d_t$ 
8        $\Delta_t \leftarrow d_t$ 
      // We cannot cancel more requests than there are
9 return  $\Delta$ 

```

E Algorithme d'extraction du scénario optimal (KCU)

Algorithm 6: Greedy Extraction of the Optimal Scenario (KCU)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Tolerance ε

Ensure: Partial graph $\mathcal{G}_{sub}$ containing the optimal scenario

```

1  $\mathcal{G}_{sub} \leftarrow$  Initialize an empty graph
2  $j_{curr} \leftarrow -1$ 
3 for  $i = 0$  to  $\Gamma$ 
4   if  $|\pi[T][i] - ub\_cost| < \varepsilon$ 
5      $j_{curr} \leftarrow i$  // Search for the first optimal terminal node
6     break
7 if  $j_{curr} \neq -1$ 
8   Add the arc  $(T, j_{curr}) \rightarrow S_{end}$  in  $\mathcal{G}_{sub}$ 
9   for  $t = T$  downto 1
10     $arc\_found \leftarrow \text{false}$ 
11    for  $i = 0$  to  $j_{curr}$  // Search for optimal antecedents
12
13      if The transition  $(i \rightarrow j_{curr})$  is physically valid
14        if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][0])| < \varepsilon$ 
15          Add the overstocking arc in  $\mathcal{G}_{sub}$  // Type 0
16           $j_{curr} \leftarrow i$ 
17           $arc\_found \leftarrow \text{True}$ 
18          break
19        if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][1])| < \varepsilon$ 
20          Add the sold out arc in  $\mathcal{G}_{sub}$  // Type 1
21           $j_{curr} \leftarrow i$ 
22           $arc\_found \leftarrow \text{True}$ 
23          break
24    if not  $arc\_found$ 
25      break
26 return  $\mathcal{G}_{sub}$ 

```

F Algorithmes d'extraction des scénarios optimaux (KCUD)

Algorithm 7: Extraction of the Optimal Paths (KCUD)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Tolerance ε

Ensure: Partial graph $\mathcal{G}_{sub}$ containing the two paths

```

1  $\mathcal{G}_{sub} \leftarrow \emptyset$ 
2  $start\_j\_late \leftarrow -1$ 
3  $start\_j\_early \leftarrow -1$ 
4 for  $i = 0$  to  $\Gamma$                                 // Searching for starting points at  $t = T$ 
5
6   if  $|\pi[T][i] - ub\_cost| < \varepsilon$ 
7     if  $start\_j\_late == -1$ 
8        $start\_j\_late \leftarrow i$            // First budget encountered (the lowest
9       index)
10     $start\_j\_early \leftarrow i$  // Last budget encountered (the highest index)
11
12 if  $start\_j\_late \neq -1$ 
13    $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \text{Adding the arches towards the sink } (T + 1, 0)$ 
14    $\text{Extract\_Extreme\_Path}(start\_j\_late, \text{"upward"}, \mathcal{G}_{sub})$  // See Algorithm 8
15   if  $start\_j\_late \neq start\_j\_early$  // Avoid recalculating if the points
16     are =
17      $\text{Extract\_Extreme\_Path}(start\_j\_early, \text{"descending"}, \mathcal{G}_{sub})$ 
18
19 return  $\mathcal{G}_{sub}$ 

```

Algorithm 8: Configurable Reporting Function

```

1 Function Extract_Extreme_Path( $j_{start}$ , direction,  $\mathcal{G}_{sub}$ )
2    $j_{curr} \leftarrow j_{start}$ 
3   for  $t = T$  downto 1
4      $arc\_found \leftarrow \text{False}$ 
5     Define the loop according to direction :  $0 \rightarrow j_{curr}$  or  $j_{curr} \rightarrow 0$ 
6     for each  $i$  in the loop
7       if The transition  $(i \rightarrow j_{curr})$  is physically valid
8         if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][0])| < \varepsilon$ 
9            $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \{(t-1, i) \xrightarrow{0} (t, j_{curr})\}$  // Type 0
10           $j_{curr} \leftarrow i$ ,  $arc\_found \leftarrow \text{True}$ 
11          break
12        if  $|\pi[t][j_{curr}] - (\pi[t-1][i] + c[t][i][j_{curr}][1])| < \varepsilon$ 
13           $\mathcal{G}_{sub} \leftarrow \mathcal{G}_{sub} \cup \{(t-1, i) \xrightarrow{1} (t, j_{curr})\}$  // Type 1
14           $j_{curr} \leftarrow i$ ,  $arc\_found \leftarrow \text{True}$ 
15          break
16     if not  $arc\_found$ 
17       break

```

G Algorithme récursif d'extraction des scénarios (DFS) (KCRDK, KCRDKL, KCOG, KCOGL)

Algorithm 9: Recursive Extraction of Scenarios (DFS) (KCOG, KCOGL, KCRDK, KCRDKL)

Require: Period t , Local budget j , Dual matrix π , Costs matrix c , Nominale solution sol , Current path P_{curr} , Set of scenarios P_{all} , Exploration limit L , Tolerance ε

Ensure: P_{all} updated

```

1 if  $|P_{all}| \geq L$ 
2   return                                     // Truncation of the search space
3 if  $t == 0$ 
4    $P_{rev} \leftarrow \text{reverse}(P_{curr})$            // Righting
5    $P_{all} \leftarrow P_{all} \cup \{P_{rev}\}$ 
6   return
7 for  $i = 0$  to  $j$                                // Assessment of past budgets
8
9   if  $j \leq i + sol.\Delta t[t-1]$  and  $(t \neq 1 \text{ or } i == 0)$  // Feasibility of the
    budget transition
10
11   if  $(j - i) \leq sol.dt[t-1]$                      // Type 0
12
13   if  $|\pi[t][j] - (\pi[t-1][i] + c[t][i][j][0])| < \varepsilon$ 
14      $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 0)\}$ 
15      $\text{Extract\_Paths\_DFS}(t-1, i, \pi, c, sol, P_{curr}, P_{all}, L, \varepsilon)$ 
16      $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 0)\}$ 
17   if  $|\pi[t][j] - (\pi[t-1][i] + c[t][i][j][1])| < \varepsilon$  // Type 1
18
19    $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 1)\}$ 
20    $\text{Extract\_Paths\_DFS}(t-1, i, \pi, c, sol, P_{curr}, P_{all}, L, \varepsilon)$ 
21    $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 1)\}$ 

```

H Schéma du fonctionnement de l'algorithme *DFS*

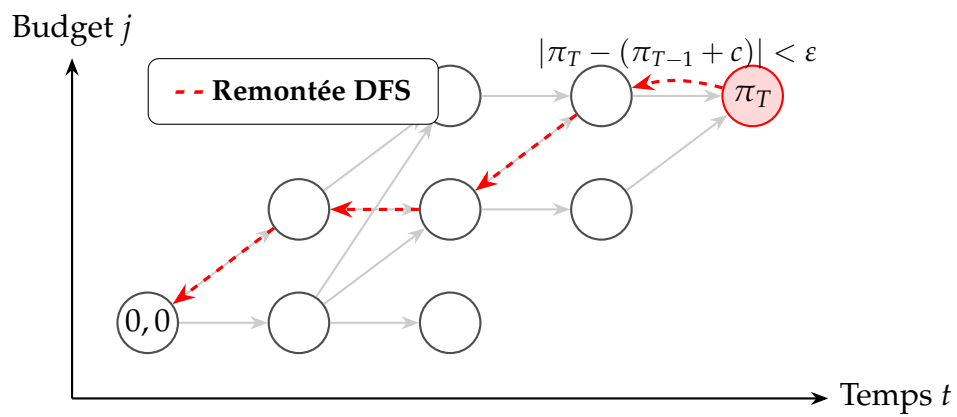


FIGURE 17 – Schéma du fonctionnement de l'algorithme *DFS*.

I Algorithme de discrétisation MaxMin (KCOG, KCOGL)

Algorithm 10: MaxMin Selection (KCOG, KCOGL)

Require: Current selection P_{sel} , Candidate pool $Pool_{cand}$, Limit on scenarios N
Ensure: P_{sel} updated with orthogonal scenarios from $Pool_{cand}$

```

1 while  $|P_{sel}| < N$  and  $Pool_{cand} \neq \emptyset$ 
2    $best\_max\_min \leftarrow -1$ 
3    $best\_candidate \leftarrow \text{null}$ 
4   for each  $p_c \in Pool_{cand}$            // Calculate the distance of the sub-set
5
6      $min\_dist \leftarrow \infty$ 
7     for each  $p_s \in P_{sel}$ 
8        $dist \leftarrow \text{Manhattan\_Distance}(p_c, p_s)$ 
9       if  $dist < min\_dist$ 
10         $min\_dist \leftarrow dist$ 
11   if  $min\_dist > best\_max\_min$            // Selection of the most distant
      candidate
12
13      $best\_max\_min \leftarrow min\_dist$ 
14      $best\_candidate \leftarrow p_c$ 
15    $P_{sel} \leftarrow P_{sel} \cup \{best\_candidate\}$ 
16    $Pool_{cand} \leftarrow Pool_{cand} \setminus \{best\_candidate\}$ 
17 return  $P_{sel}$ 

```

J Algorithme complet d'extraction (KCOGL)

Algorithm 11: Extraction of N Scenarios via L_1 -Distance (KCOGL)

Require: Dual matrix π , Cost matrix c , Optimal cost ub_cost , Limit on scenarios to push up N , Exploration limit L , Tolerance ε

Ensure: Set of scenarios P_{sel} (of maximum size N)

```

1  $sub\_OPT \leftarrow \text{Calculate\_Bound}(ub\_cost, \tau)$ 
2  $P_{opt} \leftarrow \emptyset$  // Set of optimal scenarios
3  $P_{cand} \leftarrow \emptyset$  // Set of sub-optimal scenarios
4  $P_{sel} \leftarrow \emptyset$ 
5 for  $i = 0$  to  $\Gamma$  // Phase 1: Extraction of optimal scenarios
6
7   if  $|\pi[T][i] - ub\_cost| < \varepsilon$ 
8   |    $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{opt}, L, \varepsilon)$ 
9
10  if  $P_{opt} \neq \emptyset$ 
11  |    $P_{sel} \leftarrow P_{sel} \cup \{P_{opt}[0]\}$  // Convergence guarantee
12  |    $P_{opt} \leftarrow P_{opt} \setminus \{P_{opt}[0]\}$ 
13  |   // Spatial filtering among remaining optimal scenarios
14  |    $P_{sel} \leftarrow \text{MaxMin\_Selection}(P_{sel}, P_{opt}, N)$  // See Algorithm 10
15
16  if  $|P_{sel}| < N$  // Phase 2: Relaxed extraction if  $N$  is not reached
17
18  |   for  $i = 0$  to  $\Gamma$ 
19  |   |   if  $\pi[T][i] \geq sub\_OPT$  and  $|\pi[T][i] - ub\_cost| \geq \varepsilon$ 
20  |   |   |    $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{cand}, L, \varepsilon)$ 
21  |   |   // Completion with sub-optimal scenarios
22  |   |    $P_{sel} \leftarrow \text{MaxMin\_Selection}(P_{sel}, P_{cand}, N)$ 
23
24  return  $P_{sel}$ 

```

K Algorithme d'extraction priorisé (KCRDKL)

Algorithm 12: Extraction of N Scenarios via Prioritized Random Sampling (KCRDKL)

Require: Dual matrix π , Costs matrix c , Optimal cost ub_cost , Limit on scenarios to push up N , Exploration limit L , Tolerance ε

Ensure: Set of scenarios P_{sel} (max size N)

```

1  $sub\_OPT \leftarrow \text{Calculate\_Bound}(ub\_cost, \tau)$ 
2  $P_{opt} \leftarrow \emptyset$  // Set of optimal scenarios
3  $P_{cand} \leftarrow \emptyset$  // Set of sub-optimal scenarios
4  $P_{sel} \leftarrow \emptyset$ 
5 for  $i = 0$  to  $\Gamma$  // Phase 1: Extraction of optimal scenarios
6
7   if  $|\pi[T][i] - ub\_cost| < \varepsilon$ 
8   |  $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{opt}, L, \varepsilon)$ 
9 if  $P_{opt} \neq \emptyset$ 
10 |  $P_{sel} \leftarrow P_{sel} \cup \{P_{opt}[0]\}$  // Convergence guarantee
11 |  $P_{opt} \leftarrow P_{opt} \setminus \{P_{opt}[0]\}$ 
12 |  $P_{opt} \leftarrow \text{Shuffle}(P_{opt})$ 
13 | while  $|P_{sel}| < N$  and  $P_{opt} \neq \emptyset$  // Priority filling using optimal
   | values
14 |
15 | | Transfer the first element of  $P_{opt}$  to  $P_{sel}$ 
16 if  $|P_{sel}| < N$  // Phase 2: Relaxed extraction if  $N$  is not reached
17
18 | for  $i = 0$  to  $\Gamma$ 
19 | | if  $\pi[T][i] \geq sub\_OPT$  and  $|\pi[T][i] - ub\_cost| \geq \varepsilon$ 
20 | | |  $\text{Extract\_Paths\_DFS}(T, i, \pi, c, P_{cand}, L, \varepsilon)$ 
21 |  $P_{cand} \leftarrow \text{Shuffle}(P_{cand})$ 
22 | while  $|P_{sel}| < N$  and  $P_{cand} \neq \emptyset$ 
23 | | Transfer the first element of  $P_{cand}$  to  $P_{sel}$ 
24 return  $P_{sel}$ 

```

L Analyse de la robustesse d'un plan de production par Monte Carlo

Le protocole expérimental est le même que celui de la Section 6.6.1.

	BA	BAE_{All}	BAE_{Few}
$\widehat{EAC}$	9828,48	9828,45	9829,09
$\hat{\sigma}$	39,5091	39,7193	40,1144
$CI_{95\%}$	[9827,7;9829,25]	[9827,67;9829,23]	[9828,3;9829,88]
WCS	9966,09	9971,85	9981,84
TWC	10034	10034	10034

TABLE 4 – Analyse de la robustesse d'un plan de production par la méthode de Monte Carlo (BA , BAE_{All} et BAE_{Few}). On a : EAC pour « expected average cost »; WCS pour « worst-case simulated » et TWC pour « theoretical worst-case ».

M Déroutement de l'algorithme de Benders Adverse

M.1 Déroutement de l'algorithme

Pour cette expérience, nous prendrons Δ_i inconstant $\forall i \in \llbracket 1, T \rrbracket$. Cependant, nous suggérerons que les coûts de lancement c^P sont nuls et que le bénéfice par objet vendu est strictement positif. $\hat{D}_{nominal}$ est la demande nominale cumulée. Le problème est formulé comme une minimisation des coûts, ce qui explique que les revenus soient des coûts négatifs.

M.1.1 Formulation du problème

Soit le jeu de données suivant :

$$T = 3 \quad (M.1)$$

$$\Gamma = 2 \quad (M.2)$$

$$\Delta_1 = 1 \quad (M.3)$$

$$\Delta_i = 2, \forall i \in \llbracket 2, T \rrbracket \quad (M.4)$$

$$c^I = 1\text{€} \quad (M.5)$$

$$c^B = 2\text{€} \quad (M.6)$$

$$c^P = 0\text{€} \quad (M.7)$$

$$b^P = 10\text{€} \quad (M.8)$$

$$\hat{D}_{nominal} = [10, 20, 30] \quad (M.9)$$

On obtient le graphe ci-dessous :

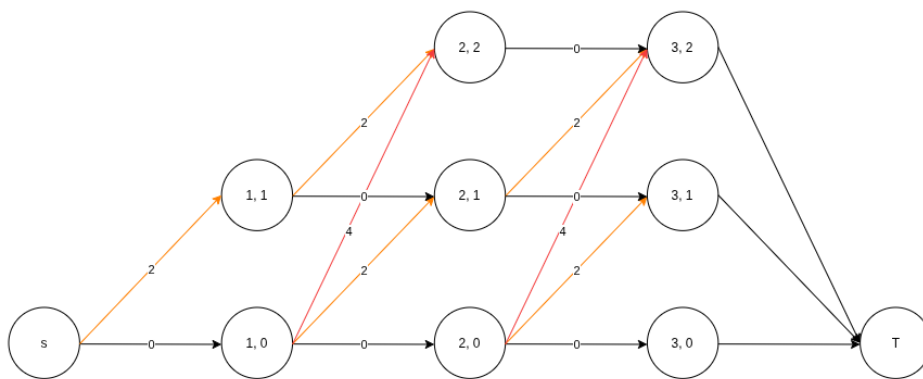


FIGURE 18 – Graphe de budget symbolisant l'expérience.

M.1.2 Itérations

Soit U l'ensemble contenant le scénario nominal et U' l'ensemble agrégé de U par les nouveaux scénarios remontés par le sous-problème.

Step 1 : Attaque nominale

Lower Bound Lors de la première étape, le problème Maître n'a aucune idée de l'incertitude ; il ne connaît que le scénario nominal. Il va donc chercher à la satisfaire.

- Plan de production cumulé : $X^{(1)} = [10, 20, 30]$;
- Pénalités = 0€ (car ni stock ni rupture) ;
- Ventes = $\sum(\max\{0, 0\}) - b^P \min\{30, 30\} = -10 \times \min(30, 30) = -300€$;
- $LowerBound = -300€$.

Upper Bound L'adversaire observe le plan $X^{(1)}$ et cherche à maximiser la fonction de coût tout en dépensant le maximum de budget possible. Nous devons calculer le coût maximum des deux scénarios (rupture/ surstock) :

Scénario 1 : Attaque D_{up} (rupture)

- $D_{up} = [11, 22, 30]$;
- Pénalités = $2 \cdot (1 + 2) = 6€$;
- Ventes = $b^P \min\{30, 32\} = 300€$;
- $UpperBound = 6 - 300 = -294€$.

Scénario 2 : Attaque D_{down} (surstock)

- $D_{down} = [10, 20, 28]$;
- Pénalités : Le Maître a produit 30. surstock de 2 objets $\times 1€ = 2€$;
- Ventes = $b^P \min\{30, 28\} = 280€$;
- $UpperBound = 2 - 280 = -278€$.

Bilan itération 1 :

$$UB(-278) > LB(-300)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{down} est remonté au Maître.

Step 2 : Contre-Attaque

On a donc $U = \{[10, 20, 30], [10, 20, 28]\}$.

Avec $b^P = 0$, les trois périodes n'ont plus la même importance. Les revenus étant comptabilisés après le dernier temps T , cette dernière période dicte la stratégie. L'adversaire va chercher à réduire sa demande pour priver le Maître de ses ventes.

Lower Bound Si a, b, c représentent la quantité de « sous-production », nous pouvons poser le plan du Maître :

$$X_1 = 10 - a \tag{M.10}$$

$$X_2 = 20 - b \tag{M.11}$$

$$X_3 = 30 - c \tag{M.12}$$

Face au scénario $D_{nom} = [10, 20, 30]$, le Maître est en rupture globale :

- Pénalités = $2 \cdot (a + b + c)$;
- Ventes = $10 \cdot (30 - c) = 300 - 10c$;
- $LowerBound_{nom} = (2a + 2b + 2c) - (300 - 10c) = 2a + 2b + 12c - 300$.

Comparons maintenant ce plan à la demande $D_{down} = [10, 20, 28]$:

- Pénalités : Rupture aux temps 1 et 2 ($a + b$), mais surstock au temps 3 ($(30 - c) - 28 = 2 - c$). Total = $2a + 2b + (2 - c)$;
- Ventes = $10 \times \min(30 - c, 28) = 280$ (en supposant $c \leq 2$);
- $LowerBound_{down} = (2a + 2b + 2 - c) - 280 = 2a + 2b - c - 278$.

Le Maître cherche l'équilibre en ces deux pires cas :

$$2a + 2b + 12c - 300 = 2a + 2b - c - 278 \implies c \approx 1.692$$

Les variables a et b s'annulent de chaque côté de l'équation. Le Maître n'a aucun intérêt à prendre des risques de rupture en début d'horizon, il fixe donc $a = 0$ et $b = 0$. Le nouveau plan est $X^{(2)} = [10, 20, 28.307]$. $LowerBound = -279.692\text{€}$.

Upper Bound Le Maître ayant réduit sa production finale pour éviter le surstock, l'adversaire attaque dans la direction opposée (à la hausse) au temps 3 : $\delta_3 = +2$.

- $D_{up} = [10, 20, 32]$;
- Pénalités : Rupture au temps 3 de $32 - 28.307 = 7.384\text{€}$;
- Ventes = $10 \times 28.307 = 283.07\text{€}$;
- $UpperBound = 7.384 - 283.07 = -275.686\text{€}$.

Bilan itération 2 :

$$UB(-275.686) > LB(-279.692)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{up} est remonté au Maître.

Step 3 : Trouver l'équilibre final

On a donc $U = \{[10, 20, 30], [10, 20, 28], [10, 20, 32]\}$.

Lower Bound Le Maître connaît les deux extrêmes de l'adversaire. En gardant le formalisme $X^{(3)} = [10 - a, 20 - b, 30 - c]$, confrontons ce plan à la nouvelle attaque D_{up} :

- Pénalités : Rupture globale. $2a + 2b + 2(32 - (30 - c)) = 2a + 2b + 4 + 2c$;
- Ventes = $10(30 - c)$;
- $LowerBound_{up} = (2a + 2b + 4 + 2c) - (300 - 10c) = 2a + 2b + 12c - 296$;

Le Maître équilibre le risque entre l'attaque D_{down} et D_{up} :

$$13c = 18 \implies c = \frac{18}{13} \approx 1.385$$

De nouveau, $a = 0$ et $b = 0$. Le plan final robuste est $X^{(3)} = [10, 20, 28.615]$. $LowerBound = -1.385 - 278 = -279.385\text{€}$.

Upper Bound Face à ce plan centré de 28.615 au temps 3, l'adversaire est bloqué :

- S'il attaque *DOWN* ($D_{down} = [10, 20, 28]$) : $UpperBound = -279.385€$;
- S'il attaque *UP* ($D_{up} = [10, 20, 32]$) : $UpperBound = -279.385€$. Toute autre combinaison d'attaque (par exemple, répartir le budget sur $t = 1$ et $t = 3$) s'avère moins efficace et laisse au Maître un profit supérieur.

Bilan itération 3 :

$$UB(-279.385) \leq LB(-279.385) + \varepsilon$$

Nous sommes à l'équilibre. L'algorithme a convergé vers la solution optimale, prouvant que le Maître garantit un bénéfice minimal de 279.385€ dans le pire des cas.

M.2 Résultats de l'expérimentation par CPLEX

```
Success : 1 files found in the instances folder.

hand_benders_instances/parsed_instances/hand_instance.dzn_parsed 2 0
10 20 28
Iteration 2 - Master Obj (LB): -279.692 | Subproblem Cost (UB): -278
10 20 32
Iteration 3 - Master Obj (LB): -279.385 | Subproblem Cost (UB): -275.692

BA-----done (Obj :-279.385)
===== END OF THE PROGRAM =====
```

FIGURE 19 – Résolution de l'instance par CPLEX via la méthode de Benders Adverse.

N Reproducing Robust Instance Scheduling

Cette annexe est en anglais pour faciliter la reproductibilité par la communauté.

N.1 Launch of the Experiment

We provide in this appendix sufficient details to reproduce the scheduling experiments. The source code is available via github at

https://github.com/mathsfrcn/STG_1RO_LAASCNRS.git. After cloning the repository¹⁴, it will be necessary to configure the *CPLEX* path in the *MakeFile*¹⁵. You can follow the instructions below:

```
# Makefile
CPLEX_DIR=/path/to/CPLEX
```

You should now be ready to run *benders_kc.cpp* by following this instruction:

```
$ make benders_kc
$ ./benders_kc
```

You can also modify the set of parameters directly in the *main* function. We have listed the directions for use in the Table 5.

Parameters	Definition
Export	
<i>bool : use_graph_export</i>	Not usable for the time being
<i>bool : use_results_export</i>	If you wish to export data
Simulation parameters	
<i>int : nb_path_to_select</i>	Max of paths to retrieve per iteration
<i>int : limit_number_path</i>	DFS Exploration limit
<i>float : eps</i>	Margin of error
<i>int : max_iter</i>	Maximum number of iterations for each pass
<i>int : max_time_s</i>	Maximum time for each pass
<i>float : p_few</i>	Probability of the <i>SelectFew</i> heuristic

Table 5 – Simulation parameters settings.

14. <http://help.github.com>

15. If you have followed the installation by default, the path must look like: *opt/ibm/ILOG/CPLEX_Studio222*. You can install the latest version of *CPLEX* by following <https://www.ibm.com/academic/>. We advise you to download the previous version to achieve better performance.

N.2 Test Different Heuristics

You can already run numerous experiments, but if you wish to create additional tests, simply follow the instructions below. Listing 1 illustrates the creation of a test comparing methods *A* and *B*.

Listing 1 – Add a comparison between two methods.

```

1 // Initialize the return variables
2 Benders_Result benders_sol_A_B;
3 float timeA, timeB;
4 float iterA, iterB;
5
6 // Initialize the output file
7 ostreamstream oss_A_B;
8 oss_A_B << folder_path << experience_name << "_A_B.csv";
9 ofstream output_A_B(oss_A_B.str());
10 // Add to the following variable
11 output_time_m_s << ".., time_master_A, time_subproblem_A, .." <<
    endl;
12
13 // Simulate the heuristic
14 A_benders_Main = heuristic();
15 B_benders_Main = heuristic();
16
17 // Add the check to the existing condition
18 if(abs(benders_sol_ref.obj_value - benders_sol_A.obj_value) >
    eps){}
19
20 // Create the analysis files
21 output_A_B << "A,"
22             << Gamma << ","
23             << tau << ","
24             << nb_path_to_select << ","
25             << limit_number_paths << ","
26             << benders_sol_A.iter << ","
27             << benders_sol_A.time << ","
28             << benders_sol_A.time_master << ","
29             << benders_sol_A.time_subproblem << endl;
30 output_A_B << "B,"..
31
32 // Add to the existing output
33 output_time_m_s << Gamma << ","
34                 << ..
35                 << benders_sol_A.time_master << ","
36                 << benders_sol_A.time_subproblem << ","
37                 << ..
38
39 // Add to the existing output
40 output_stats << "A,"
41             << Gamma << ","

```

```

42         << tau << ", "
43         << nb_path_to_select << ", "
44         << limit_number_paths << ", "
45         << benders_sol_A.time << ", "
46         << benders_sol_A.iter << ", "
47         << ..
48
49     // Close the document in its current state.
50     output_A_B.close();

```

There are two main functions for testing all the heuristics. *BA_benders_Main* simulates the Benders Adverse method, while *KC_benders_Main* simulates the KC heuristics. To select the desired KC heuristic, you must adjust the function call using one of the following parameters: *KC_Method* :: {*KC*, *RD*, *RD**KL*, *Unique*, *UniqueDual*, *HOG*, *HOGL*}¹⁶.

N.3 Instance Generation

Instances are generated using the following command. Once generated, the instances are placed directly into the *toy_instances* folder¹⁷. It contains various parameters listed in Table 6.

Parameters	Definitions
<i>str</i> : <i>outdir</i>	Discharge package
<i>int</i> : <i>nb_instances</i>	Number of instances
<i>int</i> : <i>nb_periods</i>	Number of periods
<i>int</i> : <i>nb_items</i>	Number of elements for each period
<i>bool</i> : <i>use_periodicity</i>	Use of periodicity
<i>bool</i> : <i>start_in_period</i>	Start during peak/off-peak periods
<i>int</i> : <i>timespan_period</i>	Duration of the period
<i>int</i> : <i>demand_prob</i>	Probability of an item being scheduled
<i>int</i> : <i>read_instance_rd_lb</i>	Interval to bound the capacity
<i>int</i> : <i>read_instance_rd_ub</i>	
<i>int</i> : <i>margin_settings</i>	Order of magnitude of costs

Table 6 – Instance generation settings.

```
$ python3 generate_random_instance.py
```

16. In this set of parameters, “KC” means “BAE”.

17. You can change the folder name, but you will then need to modify the path used to read the instances in *benders_kc.cpp*.

N.4 Statistical Analysis of the Results

The results are located in the *results* folder. Each experiment has its own timestamped folder containing the simulation's main parameters. For example, we have: *result_2026-07-30_1701_n=0_l=0_pfew=0.35_advmargin=2_BA_KC.csv*, where *n* represents the number of paths to backtrack, *l* is the DFS exploration limit, *p_few* is the probability of selecting an arc using the *SelectFew* heuristic, and *adv_margin* corresponds to the margin allocated to the subproblem when using the uncertainty budget Γ ; the higher this variable, the more difficult the resolution becomes. *adv_margin* = 1 means the adversary can use only $1 \times \Gamma$, making the problem easier to solve. The last two elements correspond to the heuristics tested in this CSV file.

You have various Python and R scripts available to perform different analyzes. The script *analyse_benders_stats.py* provides a clear overview of the average computation time and the average number of iterations for each simulated method. *analyse_benders_resultsv3.py* generates several graphs, offering a more detailed visualization based on various metrics. Finally, *analyse_time_results.R* allows you to visualize, among other things, the time spent on the master problem and the subproblem. All these scripts can be run using the following commands:

```
$ cd results
$ python3 analyse_benders_stats.py "folder/file.csv"
$ python3 analyse_benders_resultsv3.py "folder/file.csv"
# For the R script, you would need to import the CSV.
$ r analyse_time_results.R
```
